

GRAPH THEORY

Algo & CP Lecture Series

ANCC × CAIC

Manya Kansal

18 June 2026

GRAPH THEORY

GRAPH THEORY

$G(V, E)$

vertices

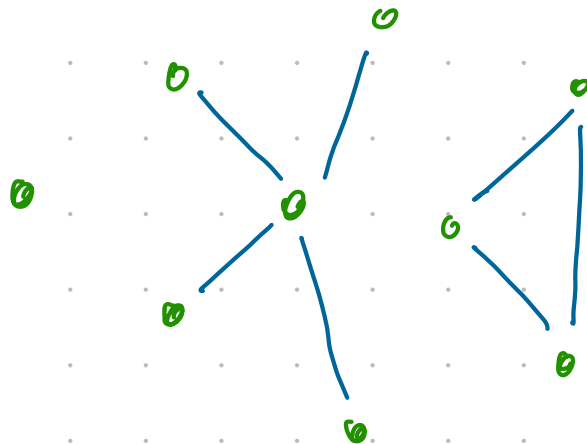
edges

GRAPH THEORY

$$G(V, E)$$

vertices

edges (undirected)

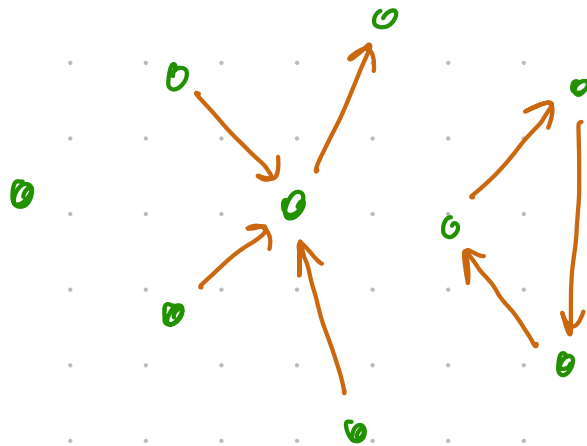


GRAPH THEORY

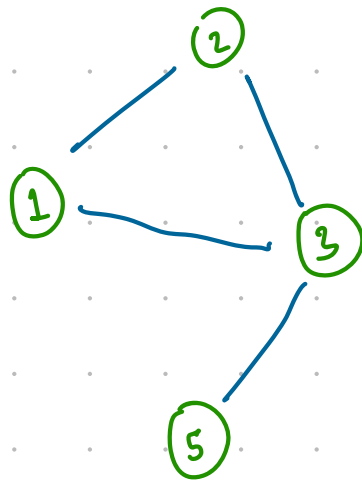
$$G(V, E)$$

vertices

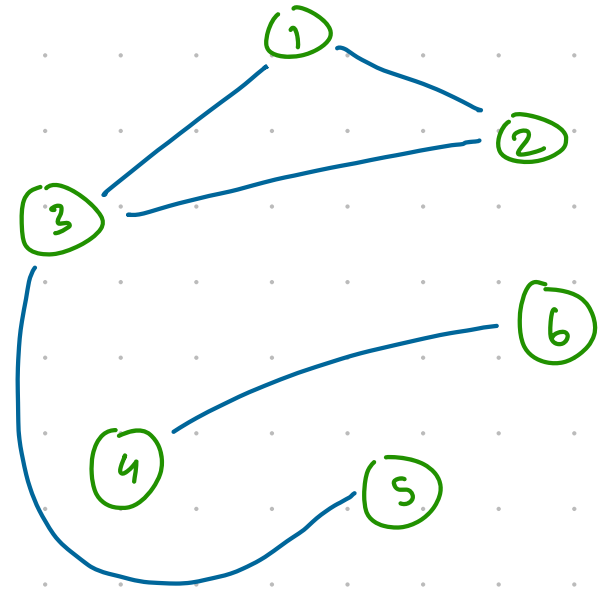
edges (directed)



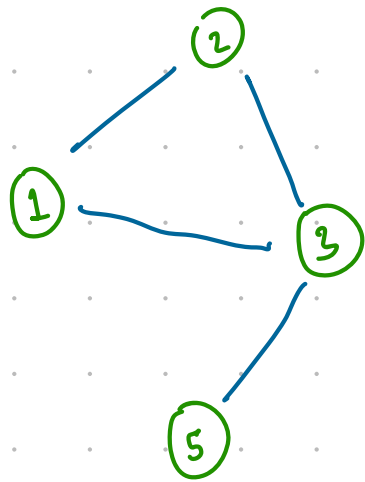
GRAPH THEORY



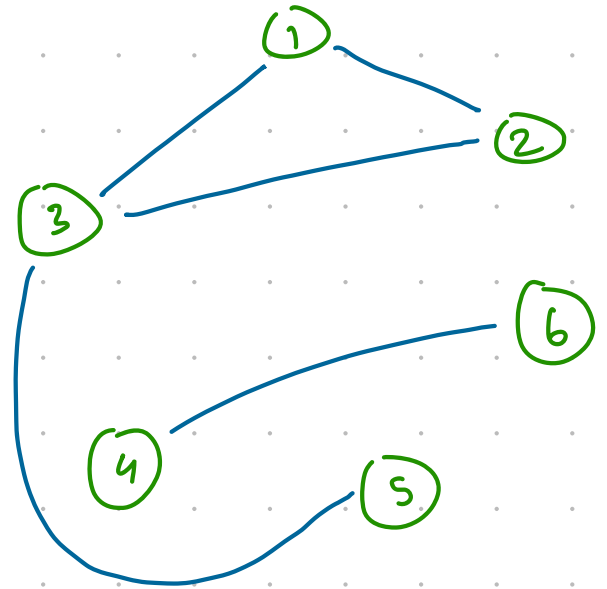
$\equiv$



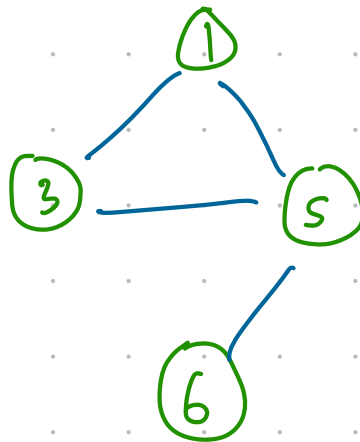
GRAPH THEORY



$\equiv$



$\neq$



unless
specified

GRAPH THEORY

We denote vertices by $v \in V$ and edges by $e \in E$ or $(u, v) \in E$.

The terms vertex and node are used interchangeably.

GRAPH THEORY

We denote vertices by $v \in V$ and edges by $e \in E$ or $(u, v) \in E$.

The terms vertex and node are used interchangeably.

SOME USEFUL TERMS

2 vertices $u, v \in V$ are adjacent if $e = (u, v) \in E$.

A path is a sequence of vertices* $(v_0, v_1, \dots, v_k)$ such that v_i and v_{i-1} are adjacent for $i = 1, 2, \dots, k$.

A cycle is a path $(v_0, v_1, \dots, v_k)$ with $v_0 = v_k$.

* unique

GRAPH THEORY

We denote vertices by $v \in V$ and edges by $e \in E$ or $(u, v) \in E$.

The terms vertex and node are used interchangeably.

SOME USEFUL TERMS

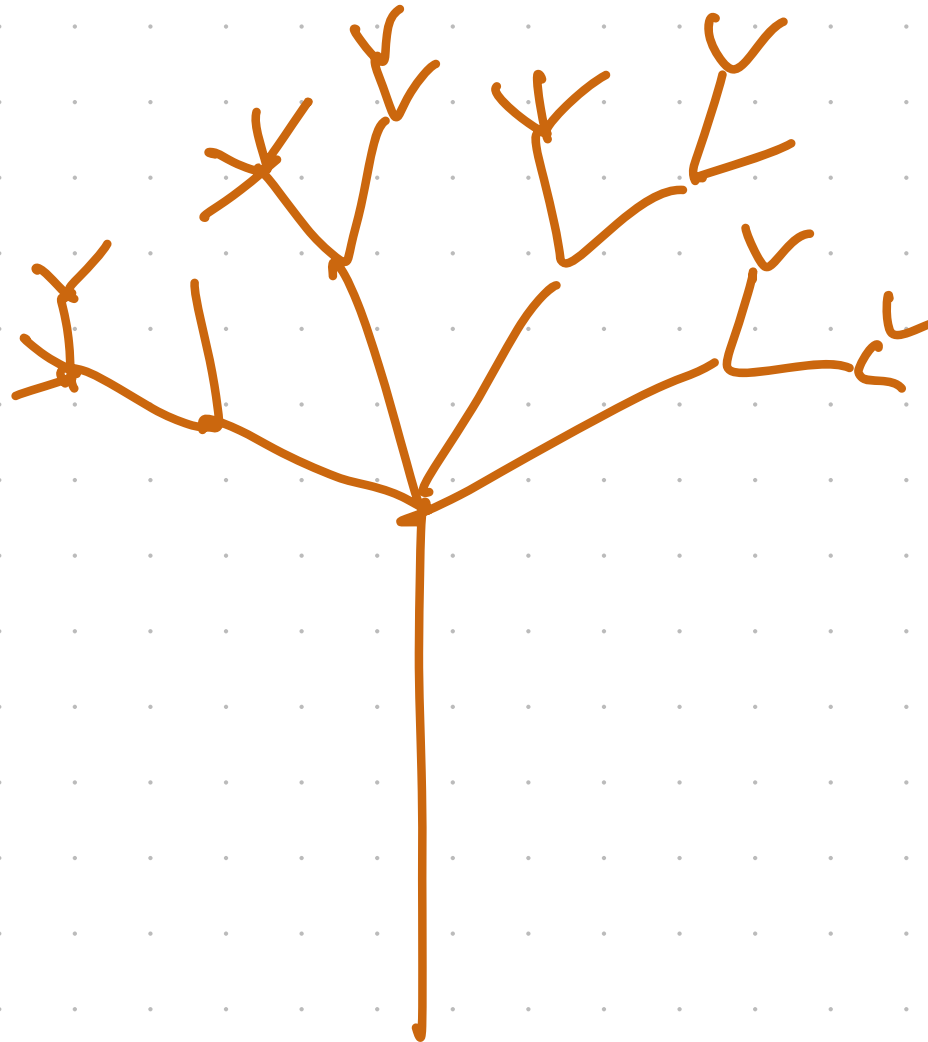
2 vertices $u, v \in V$ are adjacent if $e = (u, v) \in E$.

A path is a sequence of vertices $(v_0, v_1, \dots, v_k)$ such that v_i and v_{i-1} are adjacent for $i = 1, 2, \dots, k$.

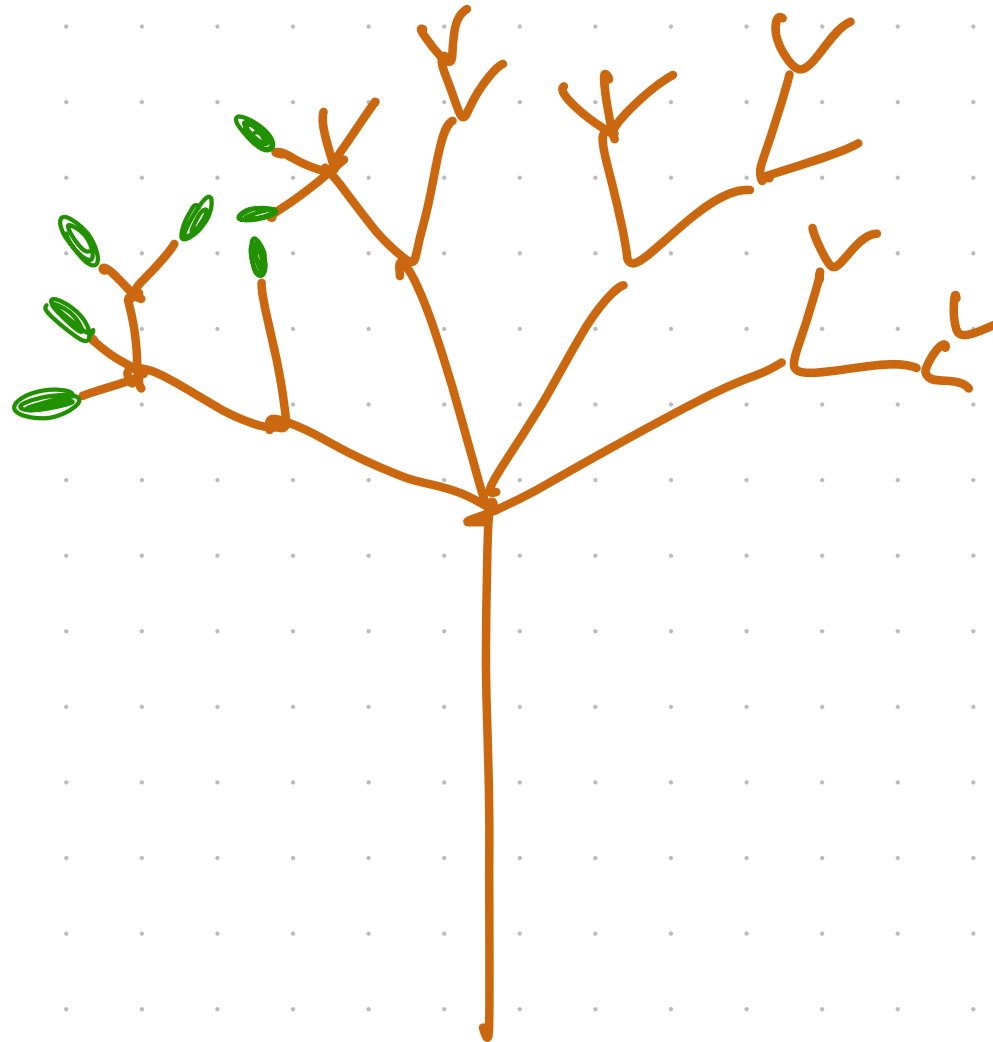
A cycle is a path $(v_0, v_1, \dots, v_k)$ with $v_0 = v_k$.

A connected component C of G is a subset $C \subseteq G$ such that $\forall u, v \in C$, there is a path from u to v & $\forall u \in C, w \in G \setminus C$ there is no path from u to w .

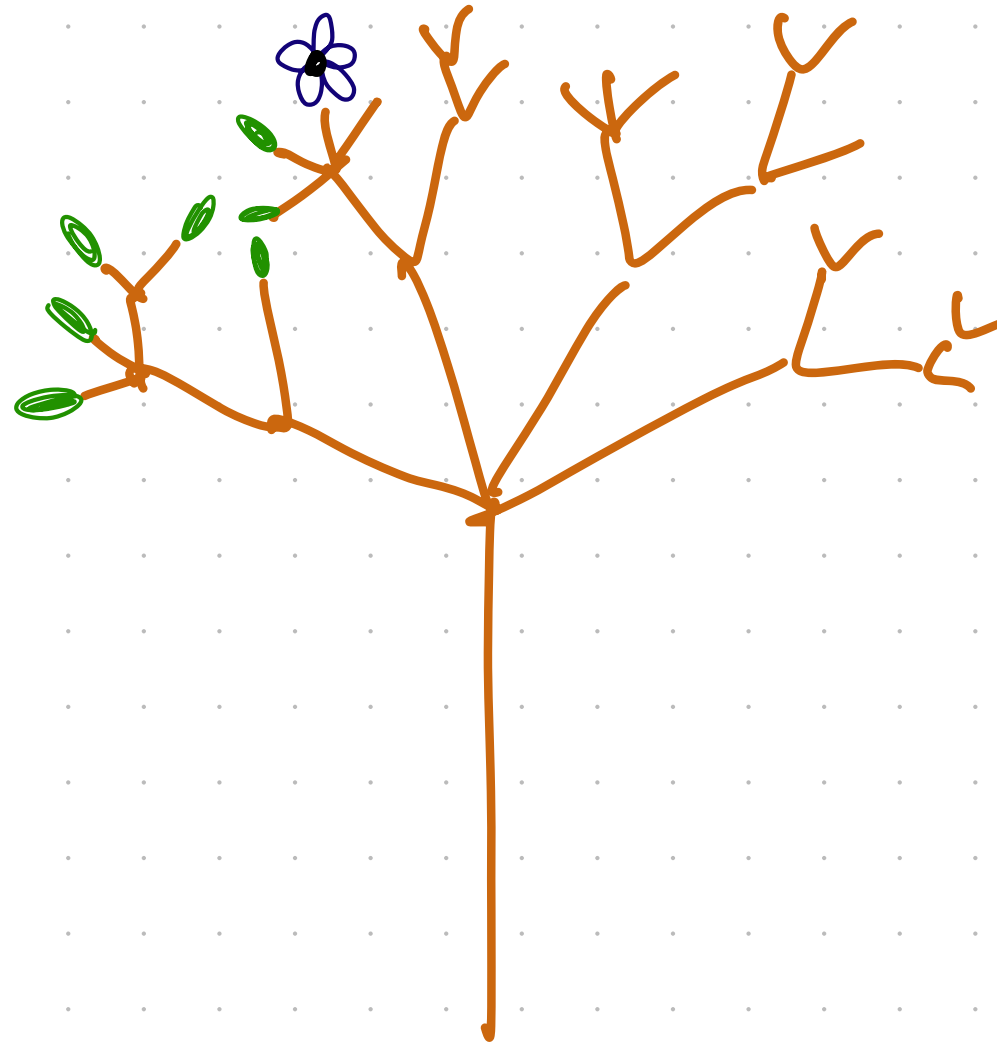
TREES



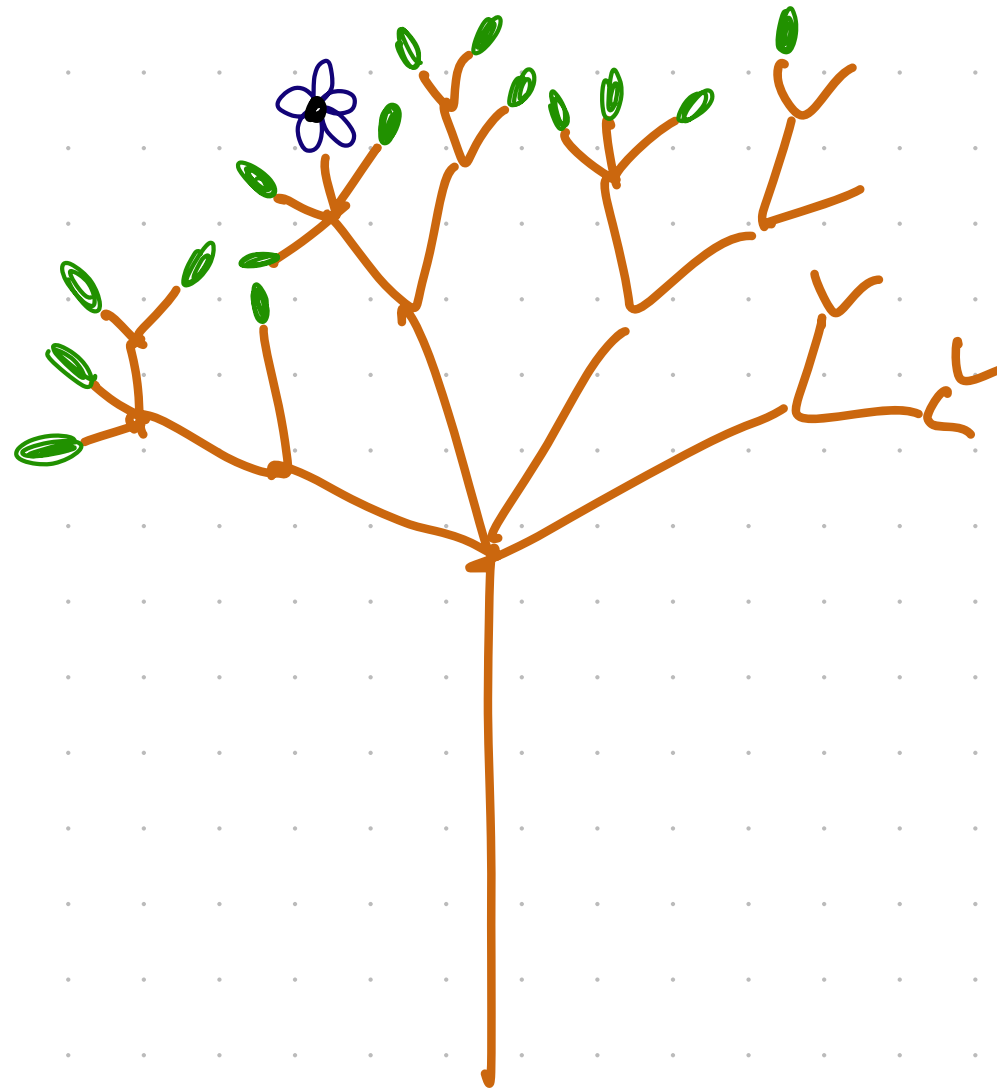
TREES



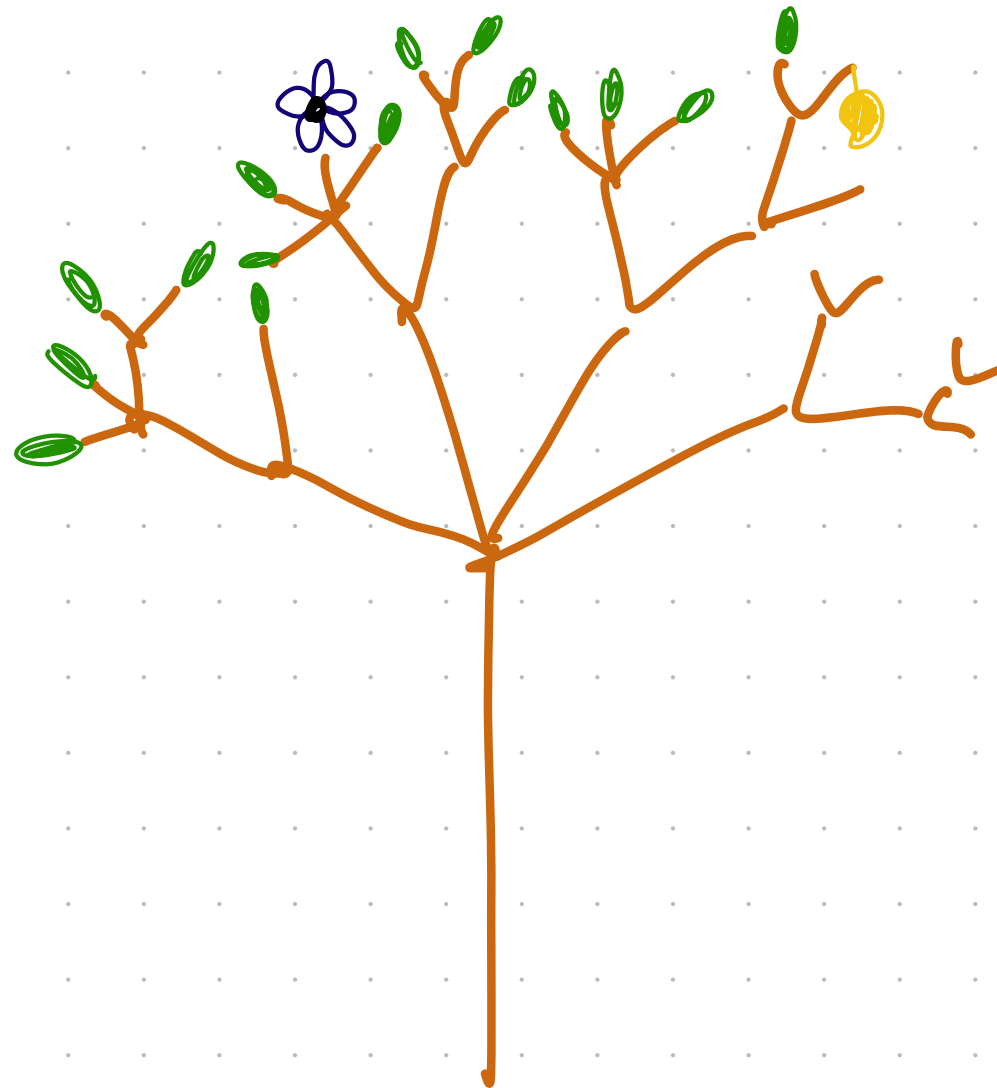
TREES



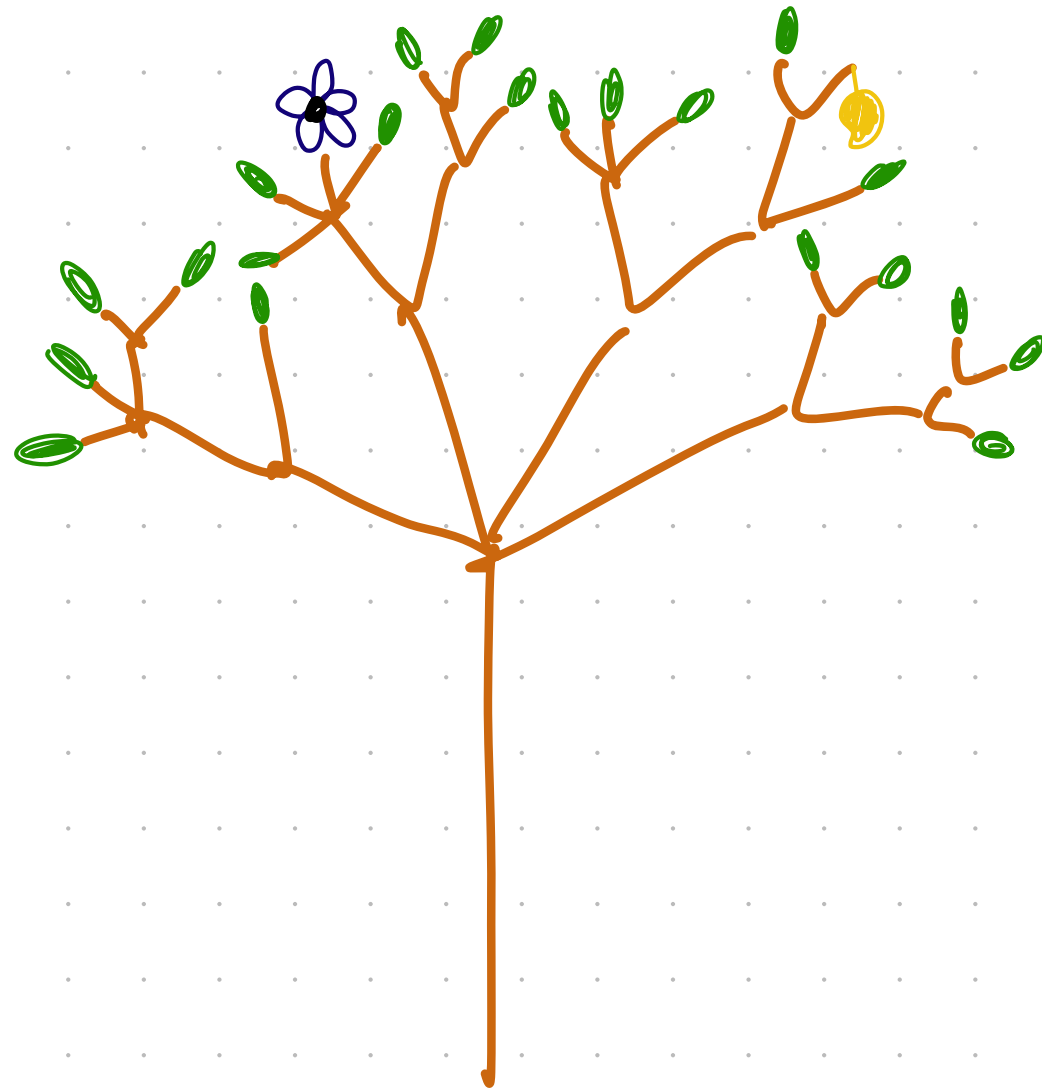
TREES



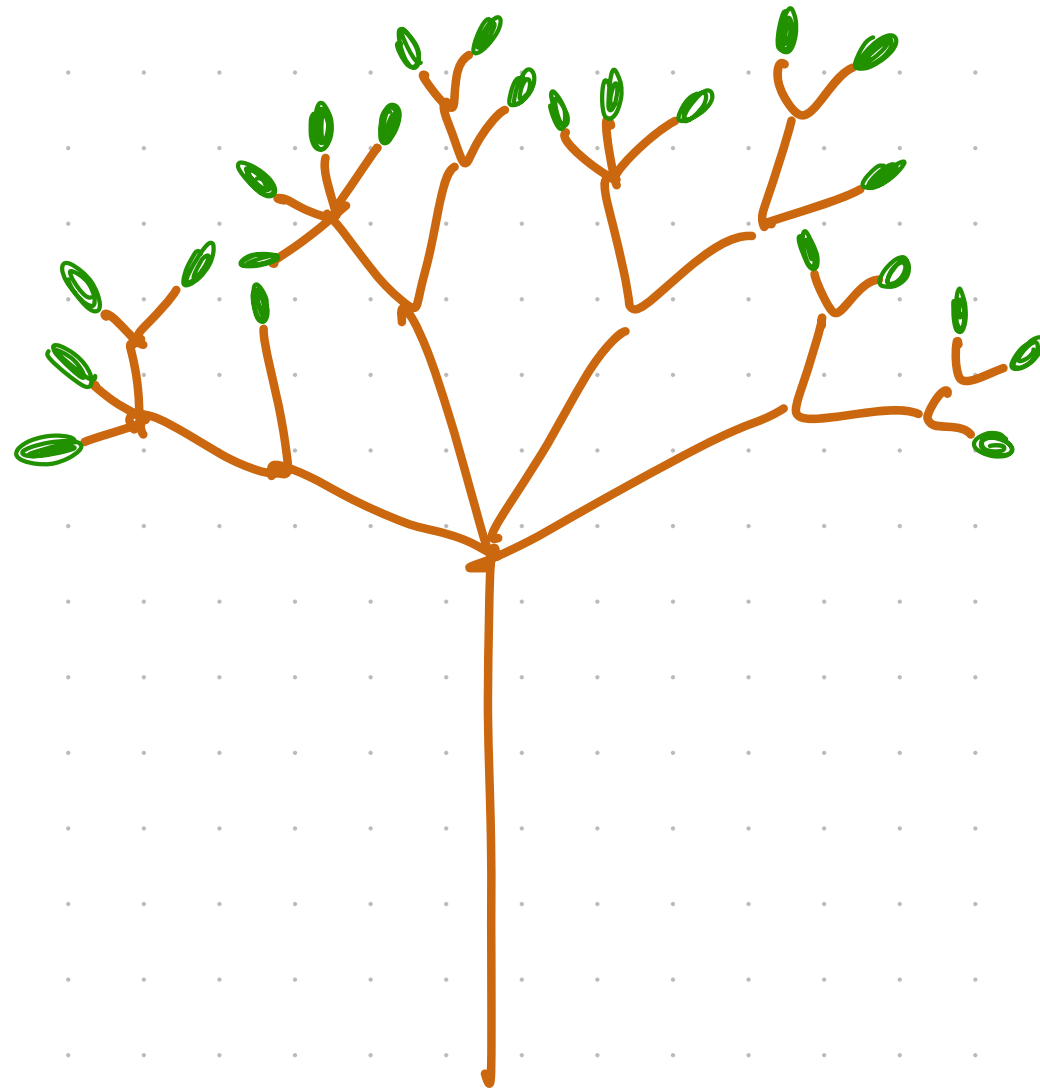
TREES



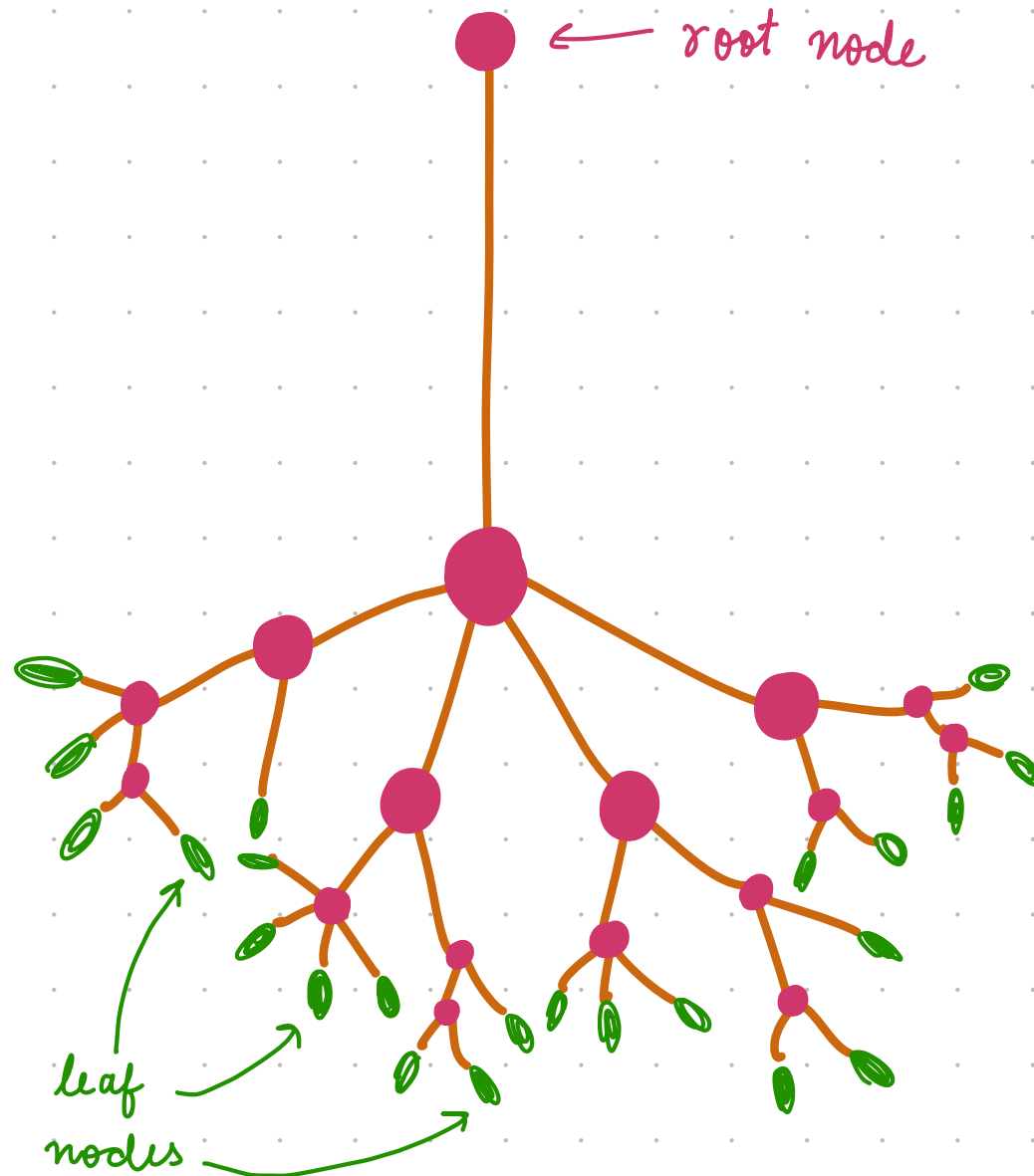
TREES



TREES



TREES



TREES

Definition : A tree is an acyclic, connected graph.

TREES

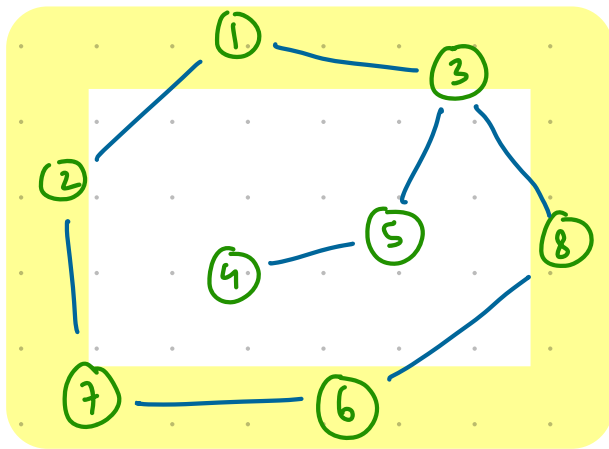
Definition : A tree is an acyclic, connected graph.

contains no cycles ← there is a path from
every vertex to every other vertex

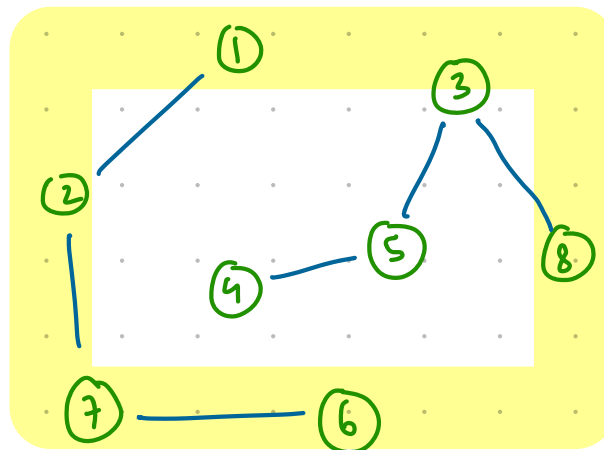
TREES

Definition : A tree is an acyclic, connected graph.

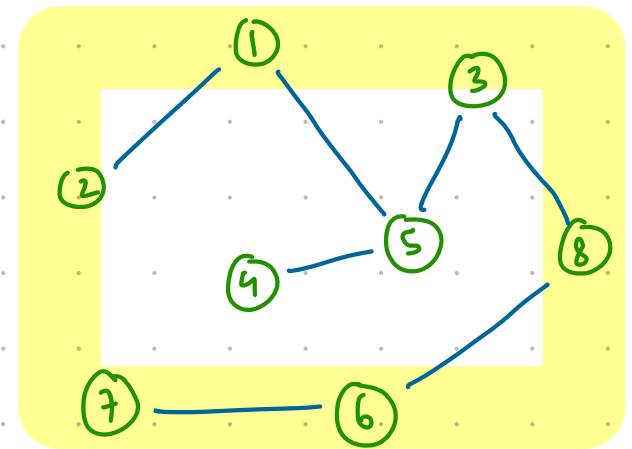
Contains no cycles ← there is a path from every vertex to every other vertex



A



B



C

Which one is NOT a tree?

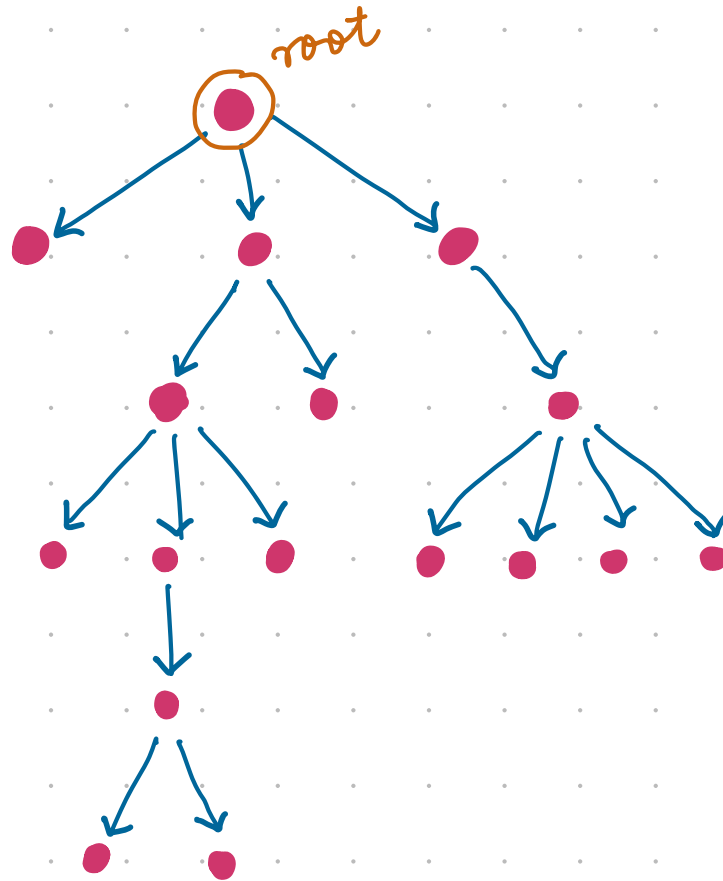
TREES

Definition : A tree is an acyclic, connected graph.

contains no cycles ← there is a path from
every vertex to every other vertex

How many edges
does a tree with n
vertices have?

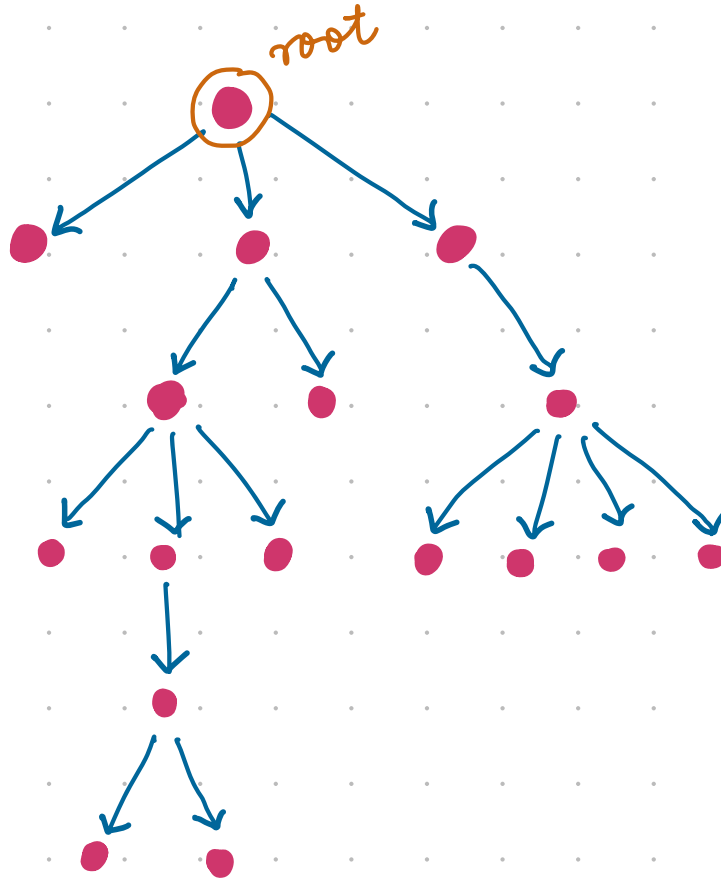
ROOTED TREES



ROOTED TREES

The depth of a node v is defined as the number of edges on the path from the root to v .

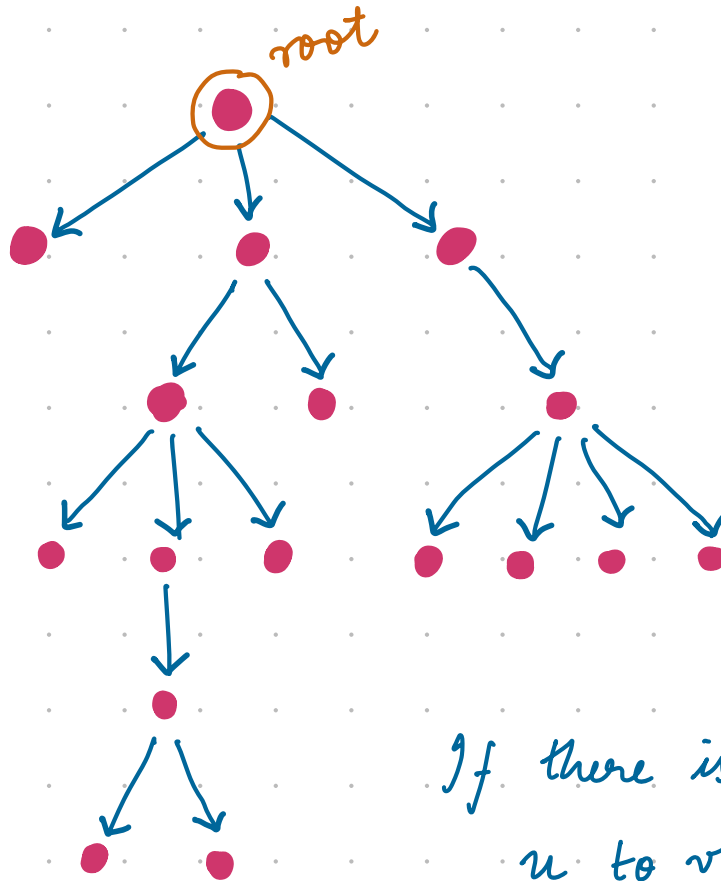
The height of a node v is defined as the maximum number of edges over all paths from v to any leaf.



ROOTED TREES

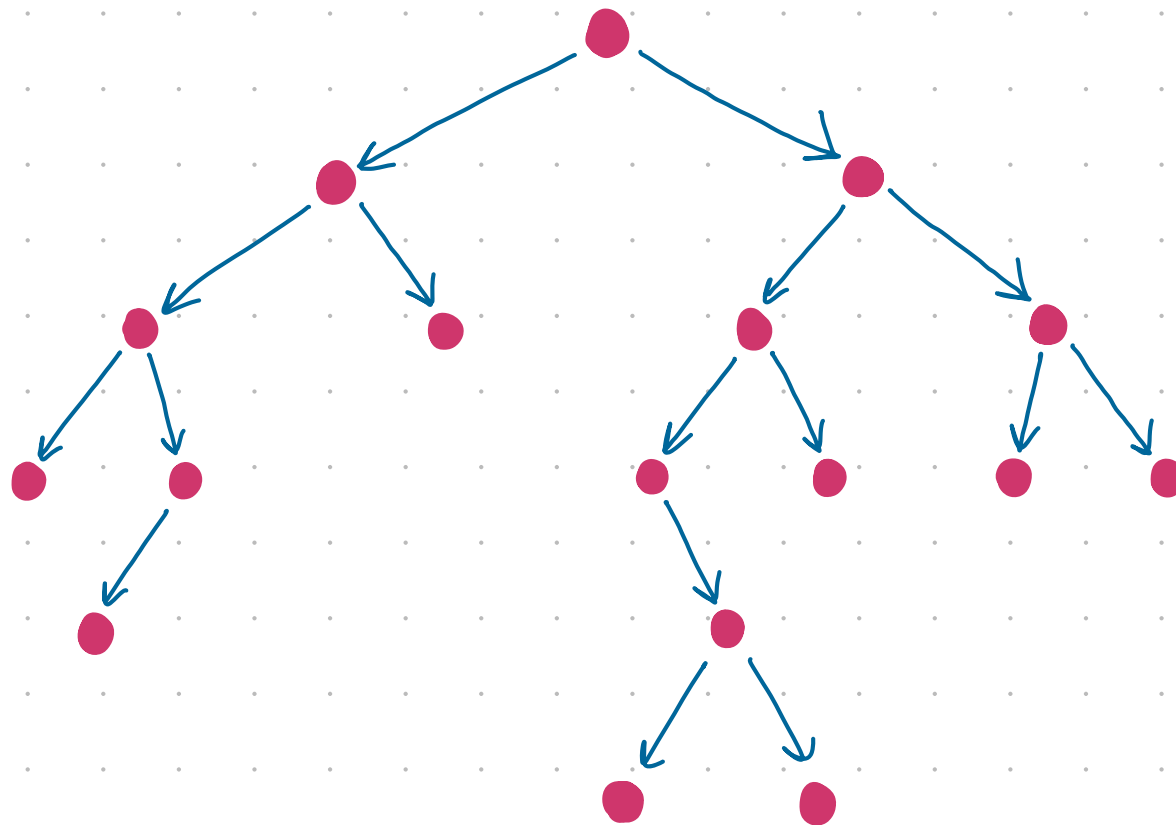
The **depth** of a node v is defined as the number of edges on the path from the root to v .

The height of a node v is defined as the maximum number of edges over all paths from v to any leaf.



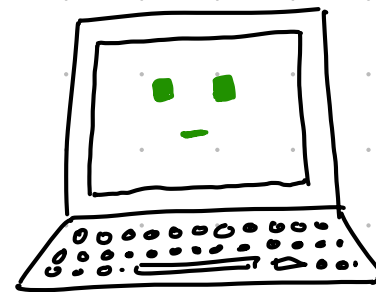
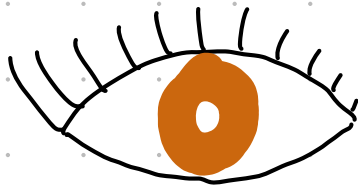
If there is an edge from u to v then u is the parent of v and v is a child of u .

BINARY TREES

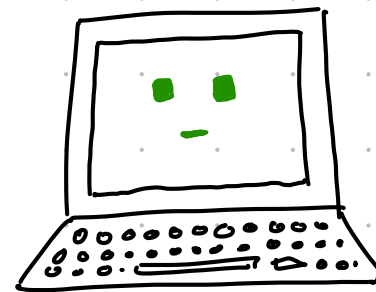
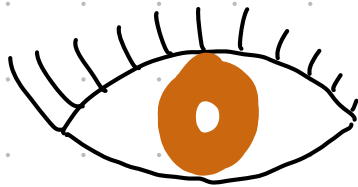
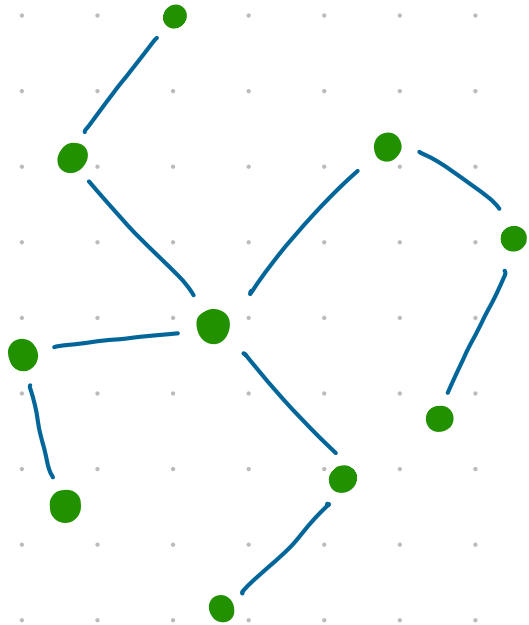


Each node
has at most
TWO
children

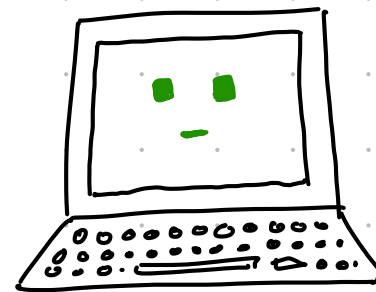
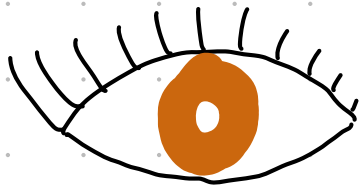
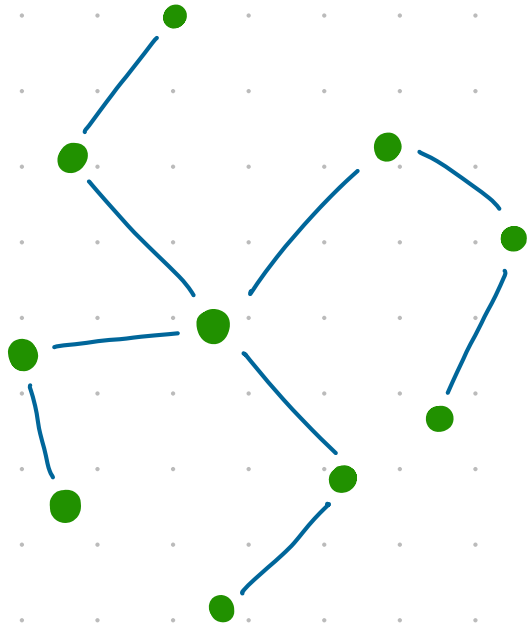
But how would a computer "see" a graph?



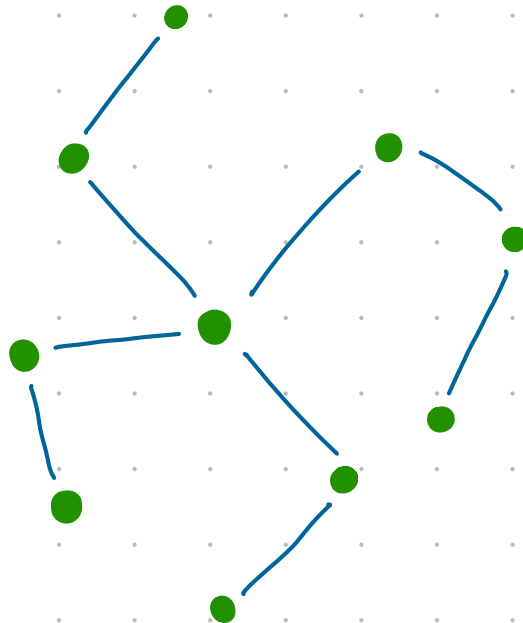
But how would a computer "see" a graph?



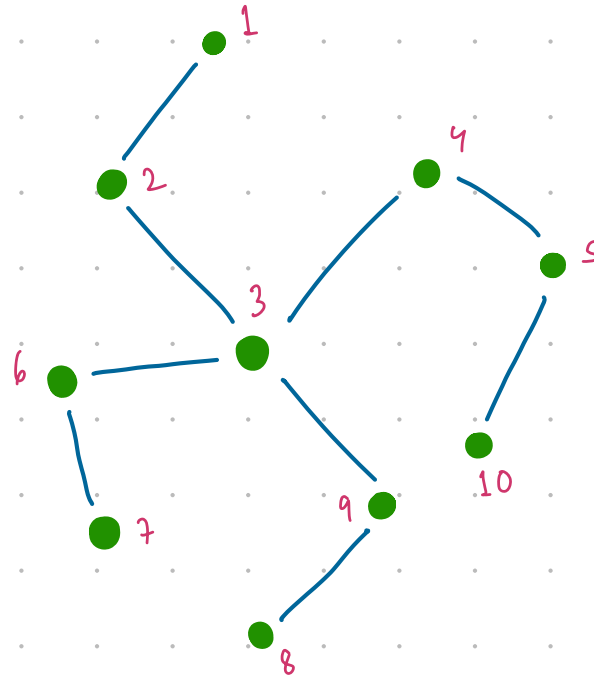
But how would a computer "see" a graph?



ADJACENCY LIST

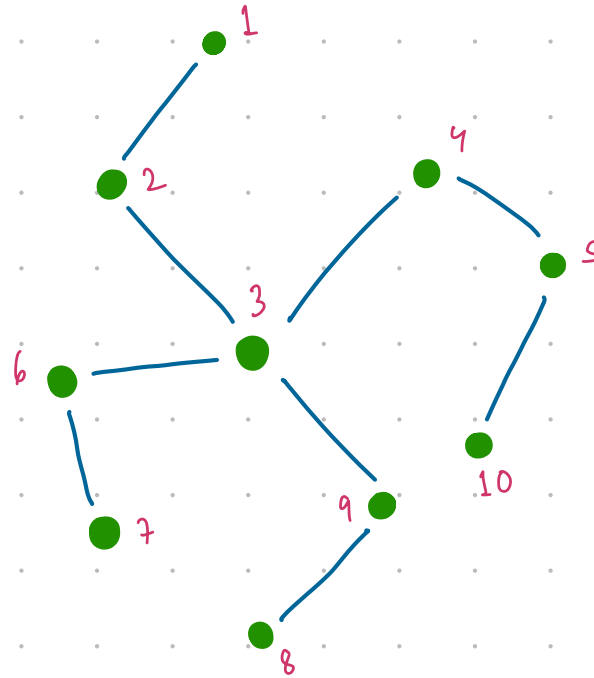


ADJACENCY LIST



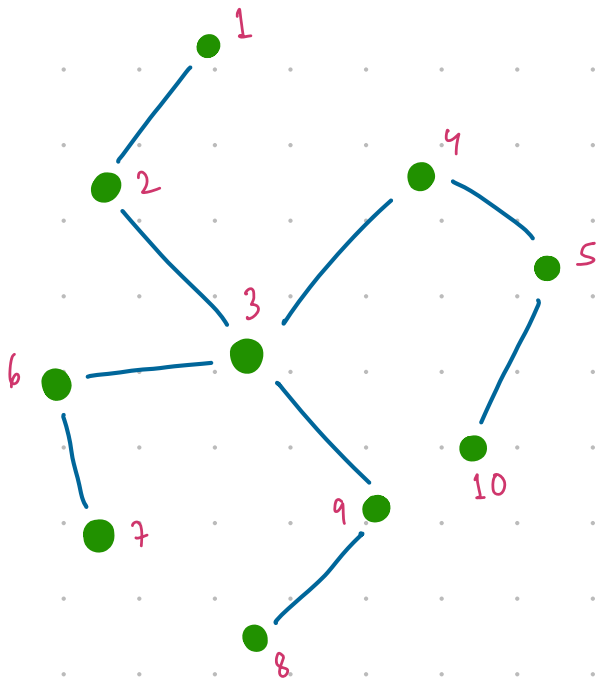
The computer doesn't see the vertices, so we need to number them in order to easily differentiate between them.

ADJACENCY LIST



Now we just store the neighbours of each vertex in a list — this makes search algorithms convenient

ADJACENCY LIST



1 : {2}

2 : {1, 3}

3 : {2, 4, 9, 6}

4 : {3, 5}

5 : {4, 10}

6 : {3, 7}

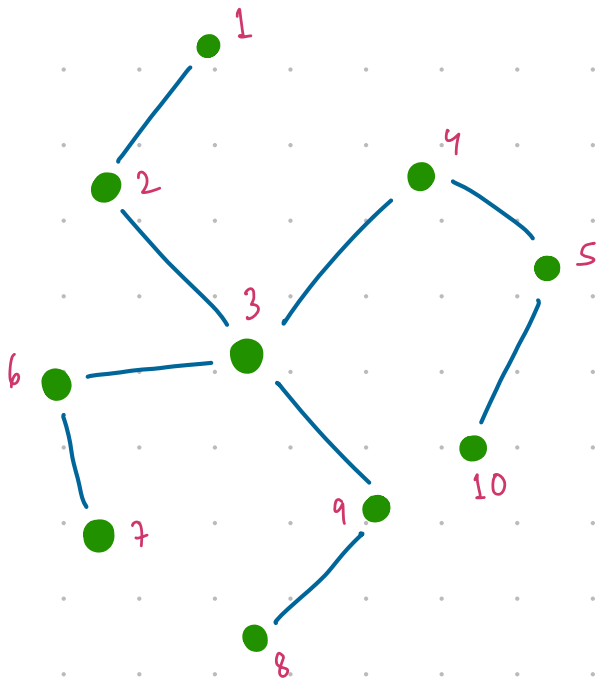
7 : {6}

8 : {9}

9 : {3, 8}

10 : {5}

ADJACENCY LIST



1 : {2}

2 : {1, 3}

3 : {2, 4, 9, 6}

4 : {3, 5}

5 : {4, 10}

6 : {3, 7}

7 : {6}

8 : {9}

9 : {3, 8}

10 : {5}

{ {2}, {1, 3}, {2, 4, 9, 6}, {3, 5}, {4, 10},
{3, 7}, {6}, {9}, {3, 8}, {5} }

ADJACENCY MATRIX

It's a matrix $A_{n \times n} = (a_{ij})$ constructed using the rule:

$$a_{ij} = \begin{cases} 1 & \text{if (there is an edge from } v_i \text{ to } v_j) \\ 0 & \text{otherwise.} \end{cases}$$

ADJACENCY MATRIX

It's a matrix $A_{n \times n} = (a_{ij})$ constructed using the rule:

$$a_{ij} = \begin{cases} 1 & \text{if (there is an edge from } v_i \text{ to } v_j) \\ 0 & \text{otherwise.} \end{cases}$$

★ for undirected graphs, A is symmetric.

★ If $P = A^k$ then p_{ij} = no. of walks from v_i to v_j with length exactly k .

* usual matrix multiplication.

ADJACENCY MATRIX

It's a matrix $A_{n \times n} = (a_{ij})$ constructed using the rule:

$$a_{ij} = \begin{cases} 1 & \text{if (there is an edge from } v_i \text{ to } v_j) \\ 0 & \text{otherwise.} \end{cases}$$

★ for undirected graphs, A is symmetric.

★ If $P = A^k$ then p_{ij} = no. of walks from v_i to v_j with length exactly k .

Proof?

* usual matrix multiplication.

Theorem: If $P = A^k$ then $p_{ij} = \text{no. of walks from } v_i \text{ to } v_j \text{ with length exactly } k$.

Proof: (By induction) Base case is $k=1$. Holds by definition.

Suppose the statement is true for $k=m$. Let $P = A^m$,

$p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } m$.

Now $Q = A^{m+1} = PA$

$$\Rightarrow q_{ij} = \sum_{r=1}^n p_{ir} \times a_{rj} = \sum_{\substack{r: \text{there is} \\ \text{a walk of length } = 1 \\ \text{from } r \text{ to } j}} p_{ir} \times 1 + \sum_{\substack{\text{remaining} \\ \text{indices}}} p_{ir} \times 0$$

(Because if there's a walk of length = 1 from v_r to v_j then $a_{rj} = 1$ otherwise $a_{rj} = 0$.)

Theorem: If $P = A^k$ then $p_{ij} = \text{no. of walks from } v_i \text{ to } v_j \text{ with length exactly } k$.

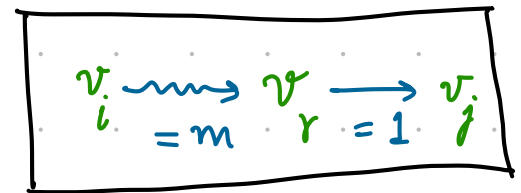
Proof: (By induction) Base case is $k=1$. Holds by definition.

Suppose the statement is true for $k=m$. Let $P = A^m$,

$p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } m$.

Now $Q = A^{m+1} = PA$

$$\Rightarrow q_{ij} = \sum_{r=1}^n p_{ir} \times a_{rj} = \sum_{\substack{r: \text{there is} \\ \text{a walk of length } = 1 \\ \text{from } r \text{ to } j}} p_{ir} \times 1$$



(Because if there's a walk of length $=1$ from v_r to v_j then $a_{rj}=1$ otherwise $a_{rj}=0$.)

Theorem: If $P = A^k$ then $p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } k$.

Proof: (By induction) Base case is $k=1$. Holds by definition.

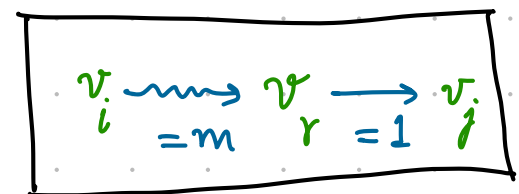
Suppose the statement is true for $k=m$. Let $P = A^m$,

$p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } m$.

Now $Q = A^{m+1} = PA$

$$\Rightarrow q_{ij} = \sum_{r=1}^n p_{ir} \times a_{rj} = \sum_{\substack{r: \text{there is} \\ \text{a walk of length } = 1 \\ \text{from } r \text{ to } j}} \left(\text{no. of walks from } v_i \text{ to } v_r \text{ with length exactly } m \right)$$

(By the induction hypothesis)



Theorem: If $P = A^k$ then $p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } k$.

Proof: (By induction) Base case is $k=1$. Holds by definition.

Suppose the statement is true for $k=m$. Let $P = A^m$,

$p_{ij} = \text{no. of paths from } v_i \text{ to } v_j \text{ with length exactly } m$.

Now $Q = A^{m+1} = PA$

$$\Rightarrow q_{ij} = \sum_{r=1}^n p_{ir} \times a_{rj} = \sum_{\substack{r: \text{there is} \\ \text{a walk of length } = 1 \\ \text{from } r \text{ to } j}} \left(\begin{array}{l} \text{no. of walks from } v_i \text{ to } v_r \\ \text{with length exactly } m. \end{array} \right)$$

$= \text{no. of walks from } v_i \text{ to } v_j \text{ with length exactly } m+1$



SEARCH ALGORITHMS

A search algorithm systematically traverses a graph i.e. visits every vertex at least once.

Two of the most common ones are depth-first search (DFS) and breadth-first search (BFS).

DFS : explore each path as far as possible

SEARCH ALGORITHMS

A search algorithm systematically traverses a graph
i.e. visits every vertex at least once.

DFS : explore each path as far as possible

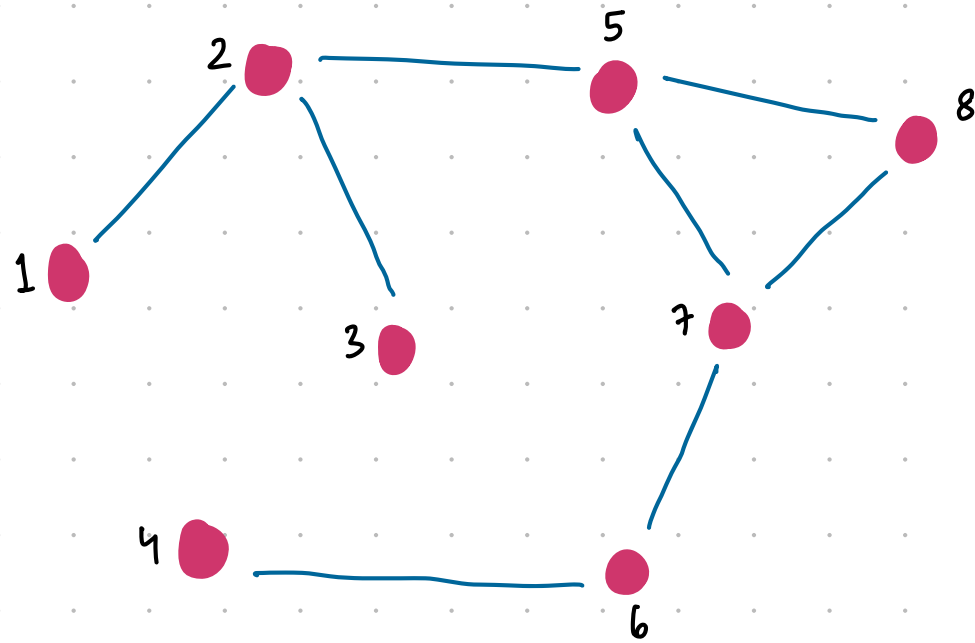
SEARCH ALGORITHMS

A search algorithm systematically traverses a graph i.e. visits every vertex at least once.

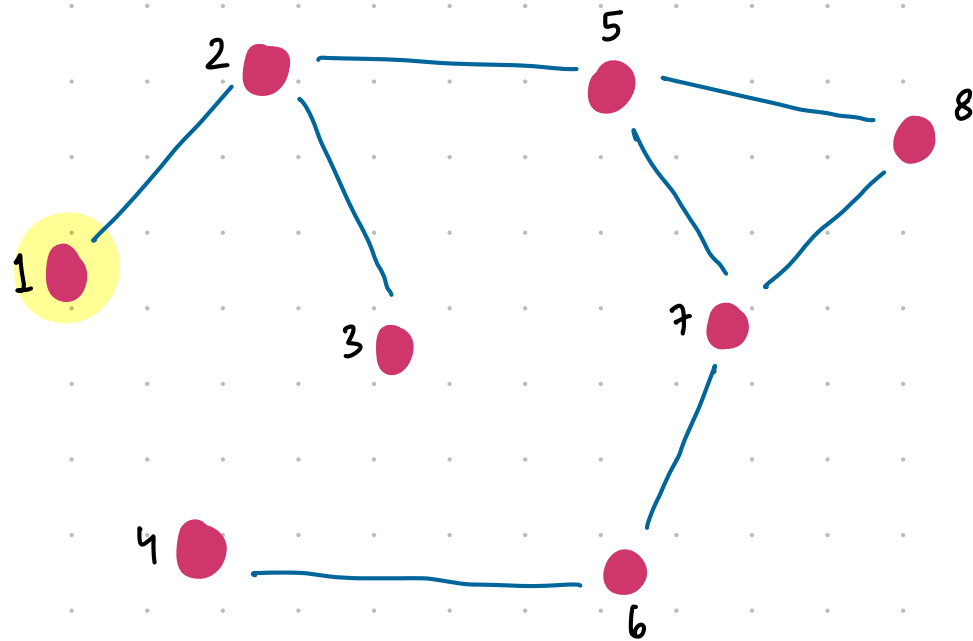
DFS : explore each path as far as possible

```
function dfs(v) {  
    if v is explored { return }  
    for each unexplored neighbour w of v {  
        dfs(w)  
    }  
    mark v explored  
}
```

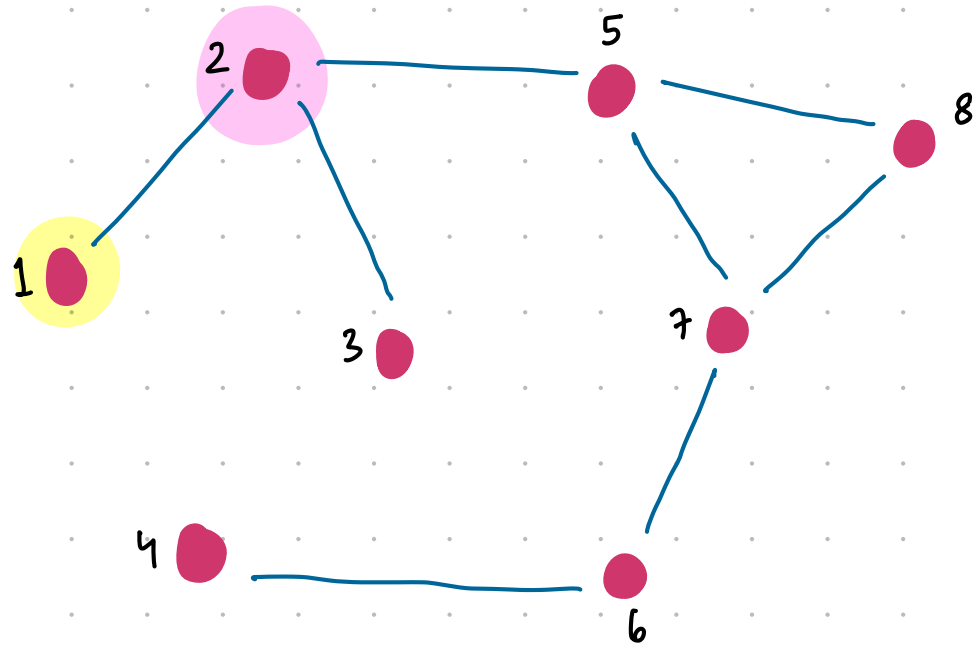
DFS



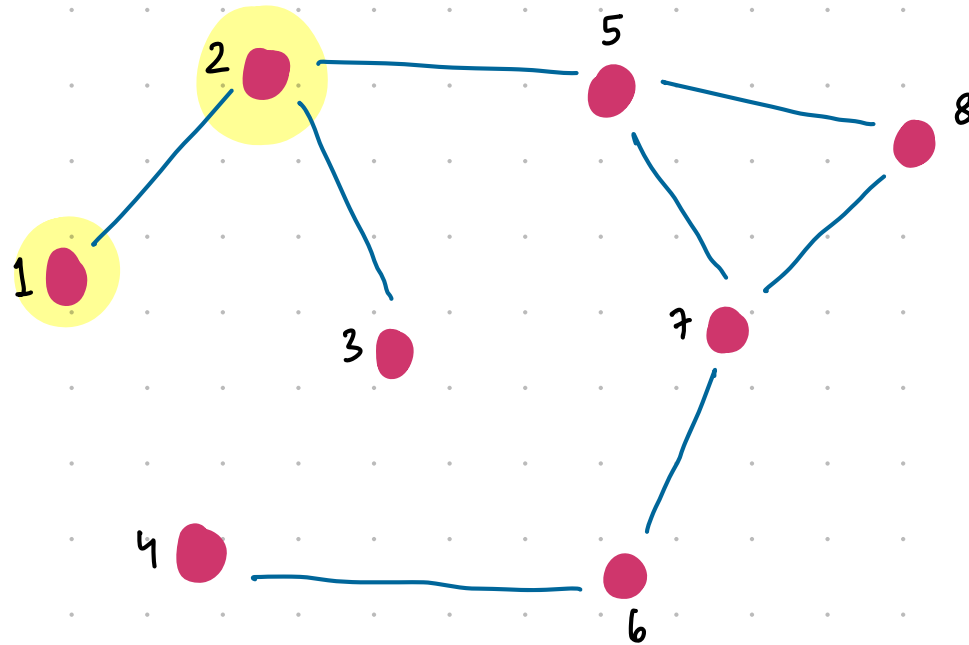
DFS (1)



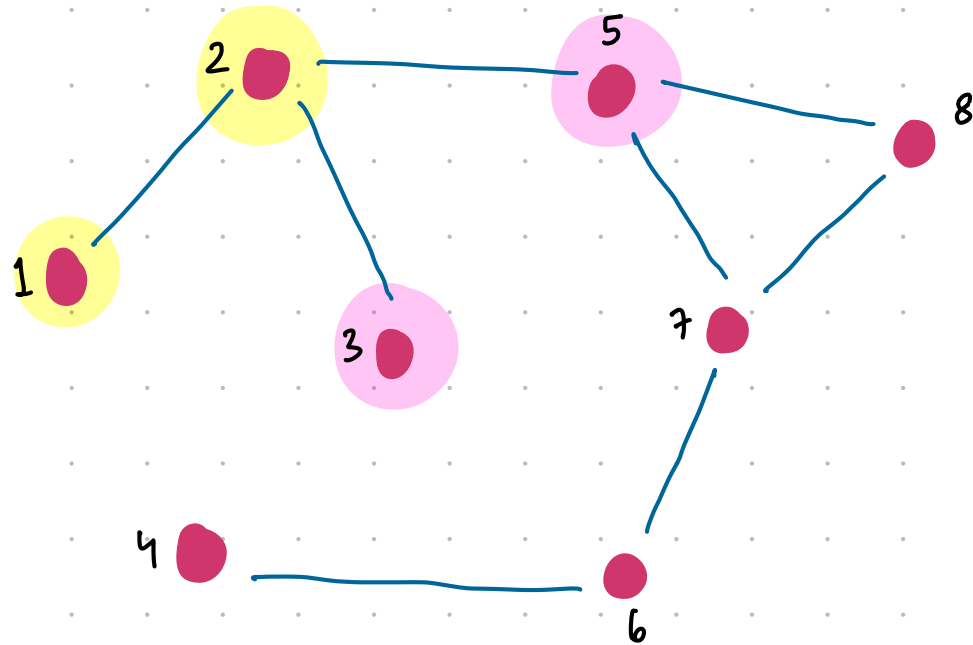
DFS



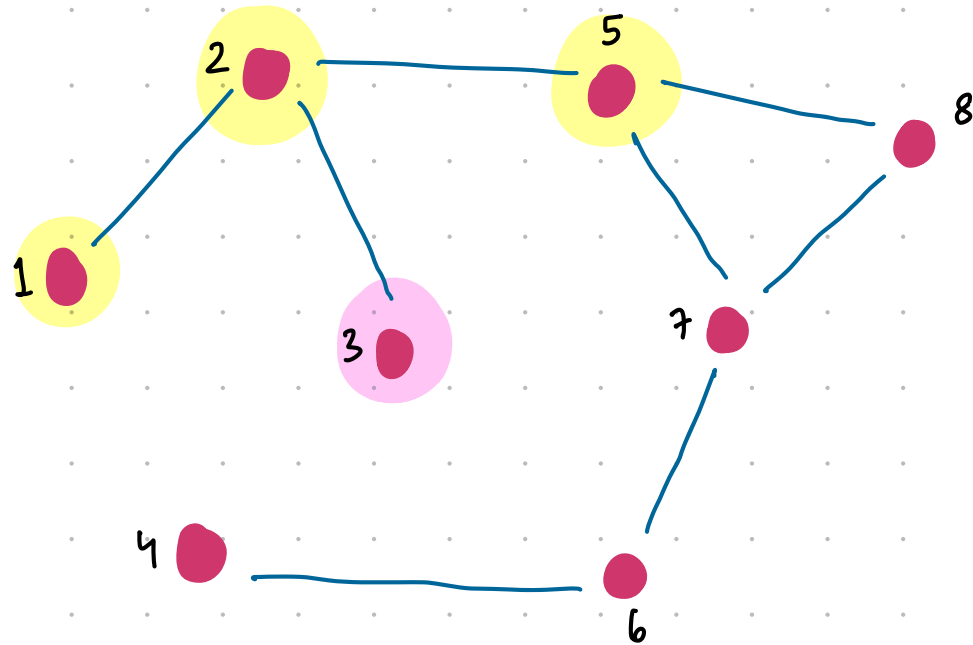
DFS



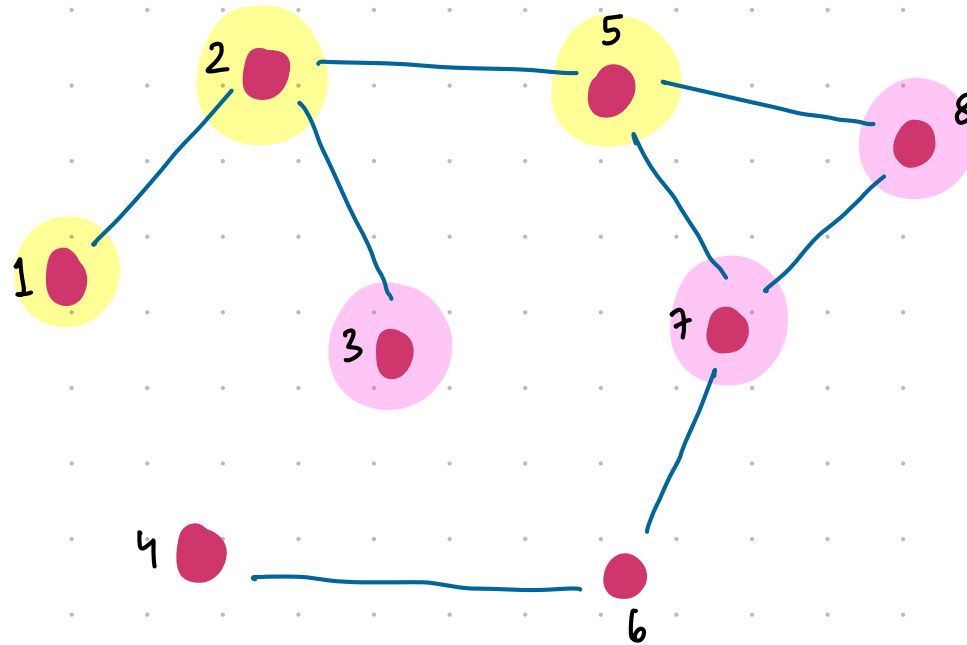
DFS



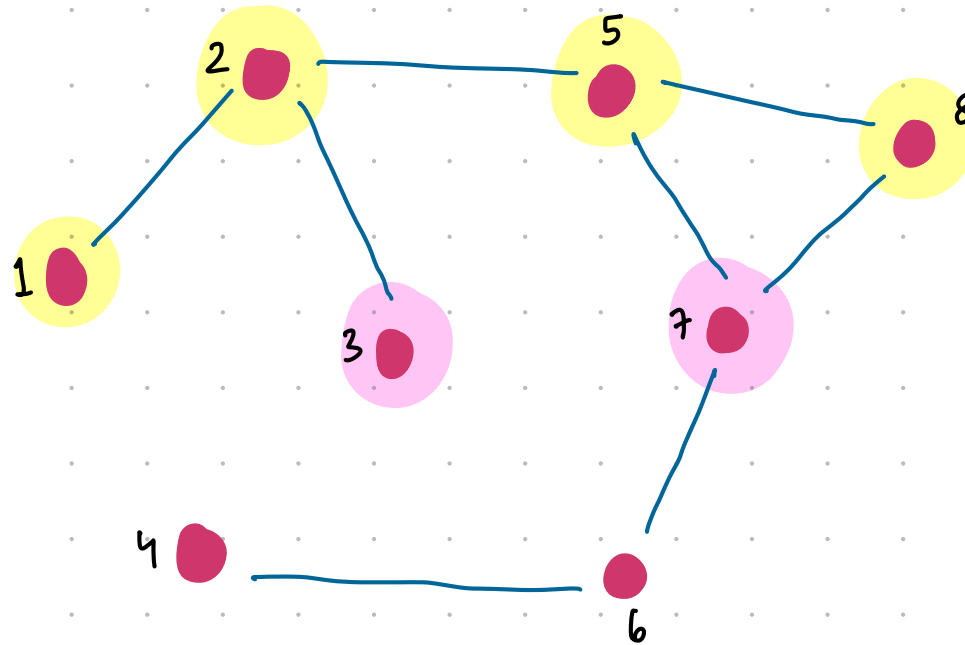
DFS



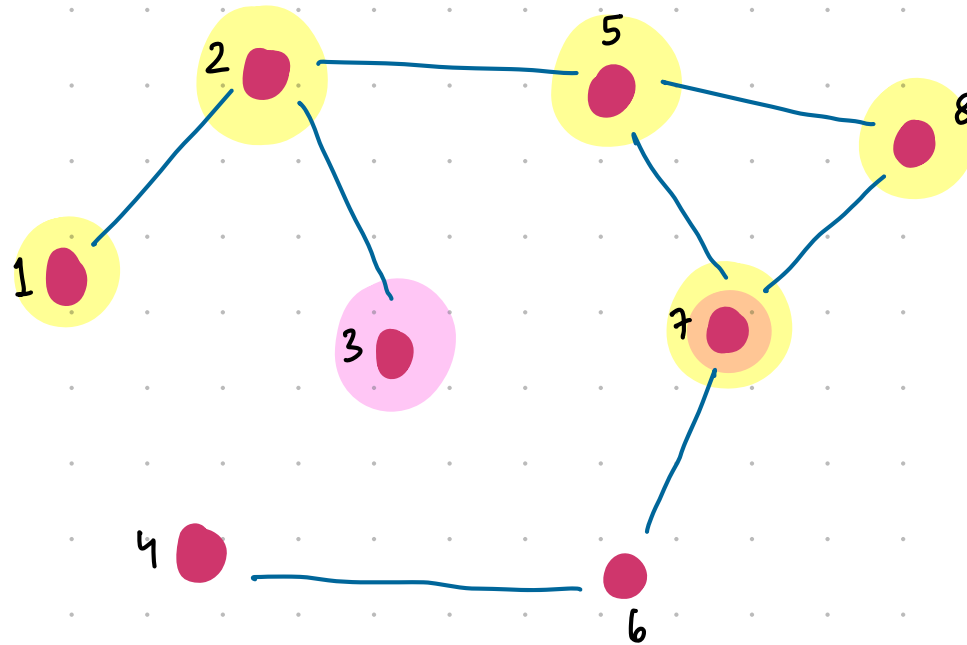
DFS



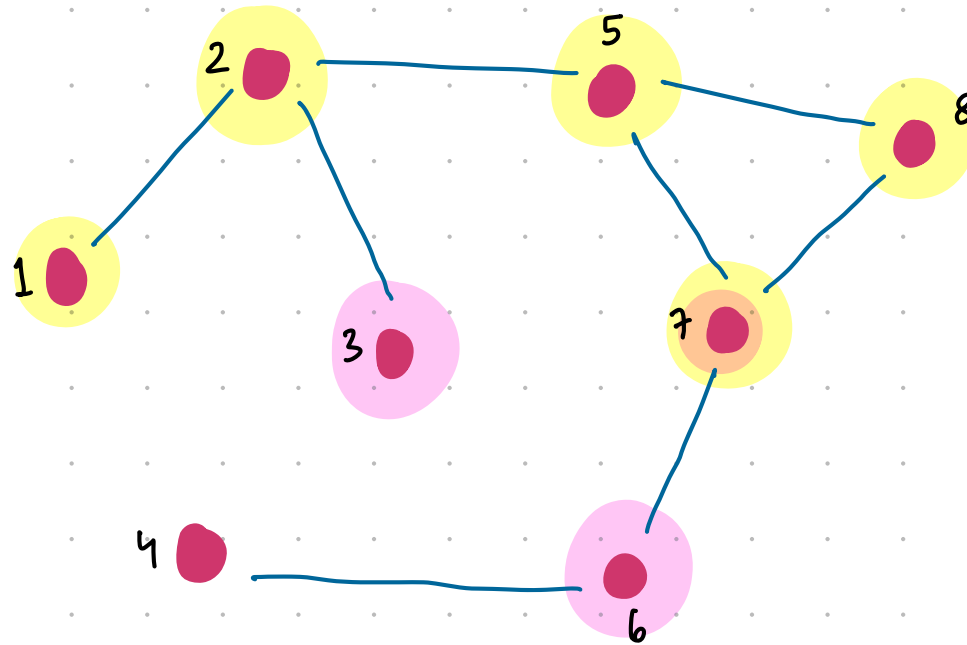
DFS



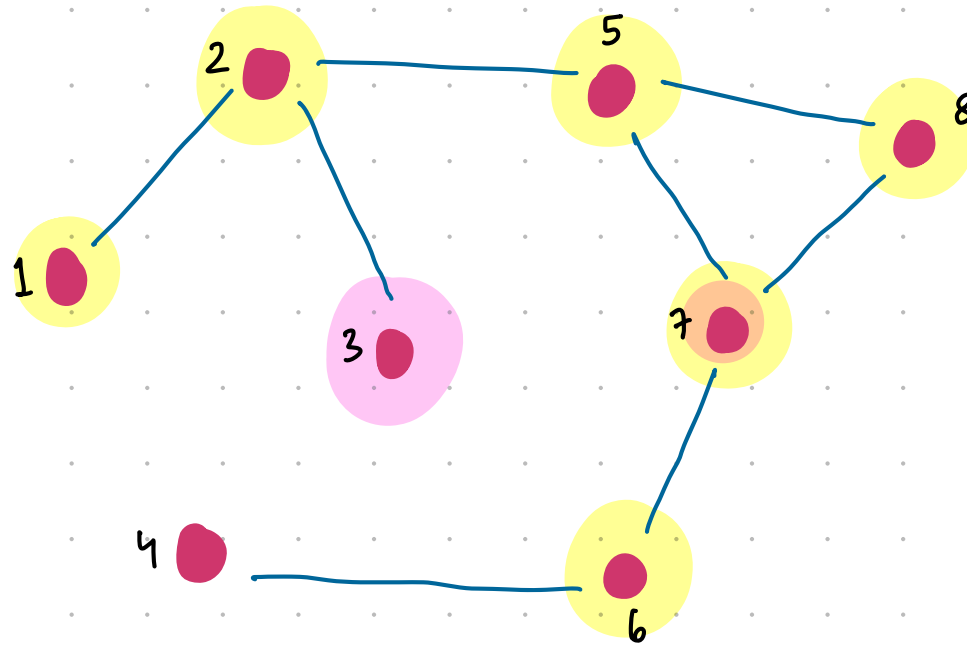
DFS



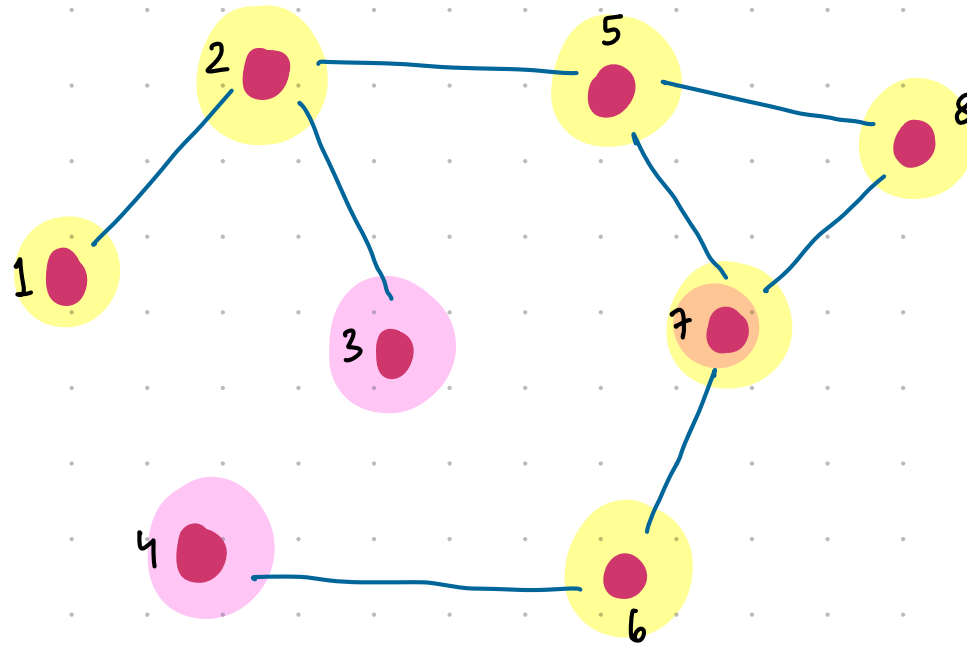
DFS



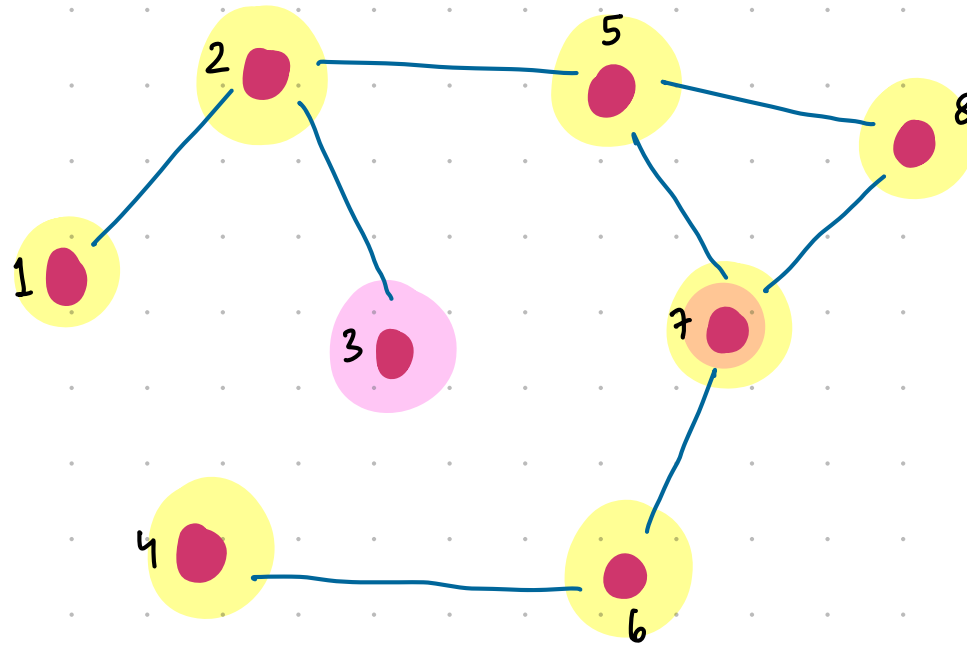
DFS



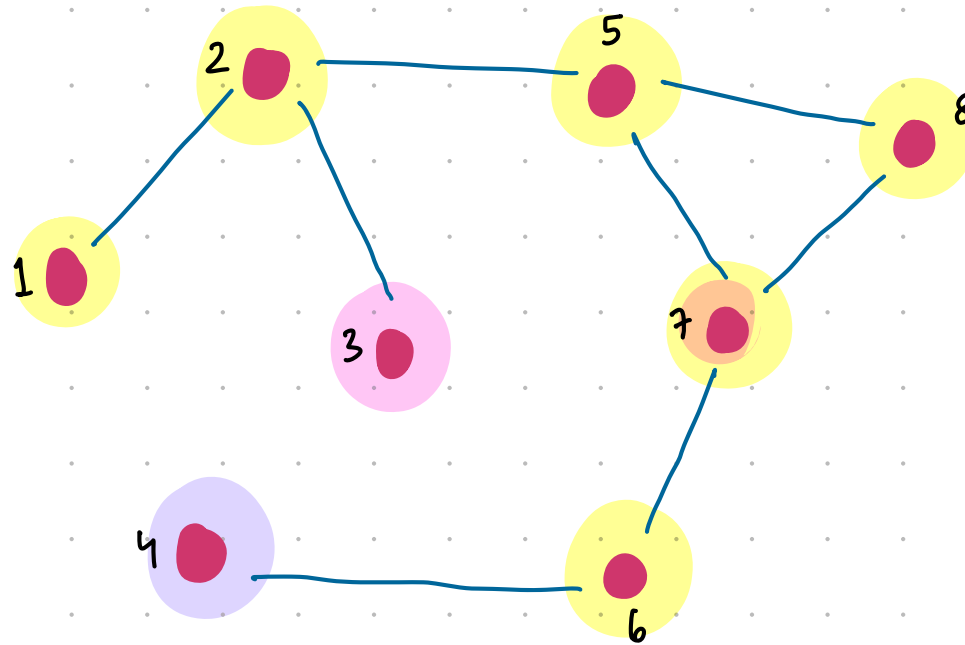
DFS



DFS

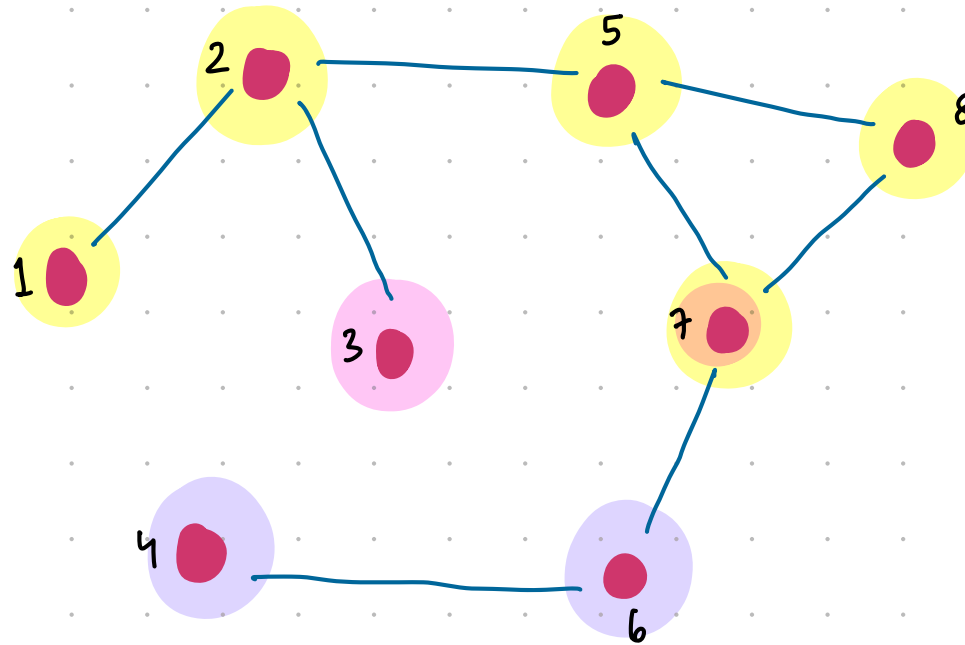


DFS



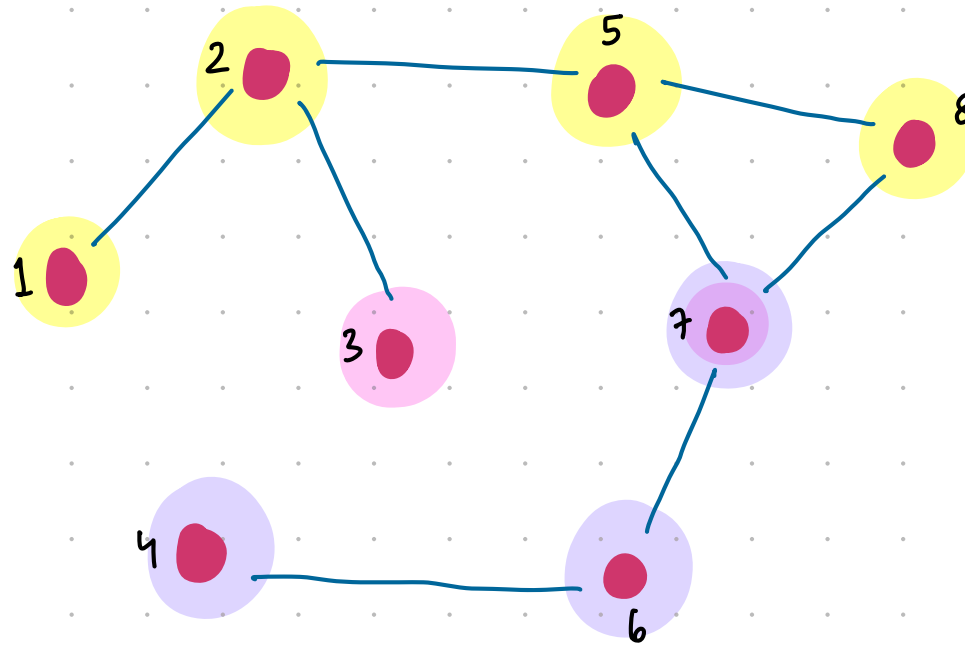
4

DFS



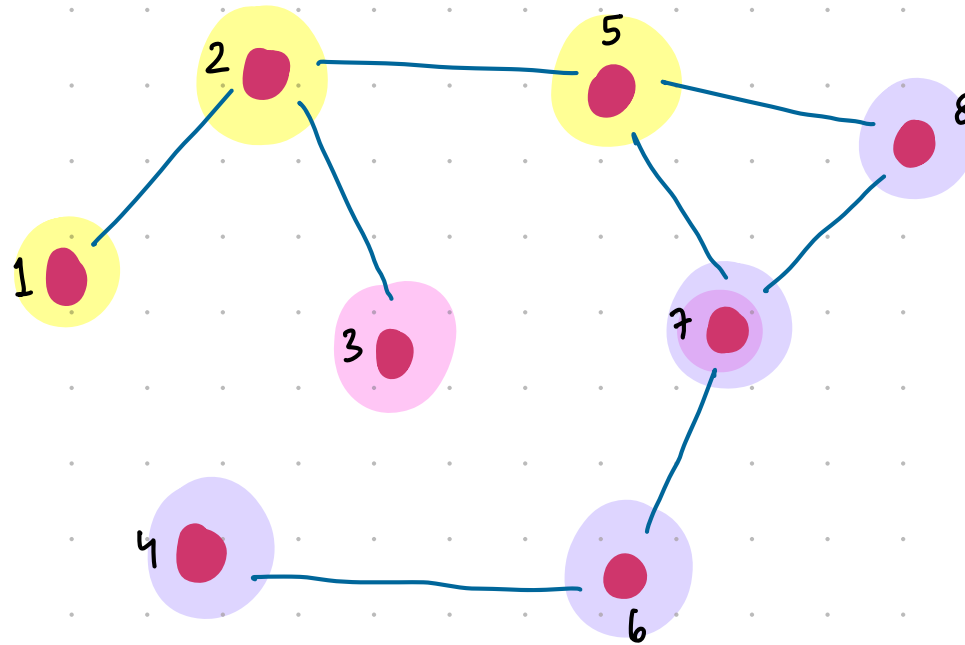
4, 6

DFS



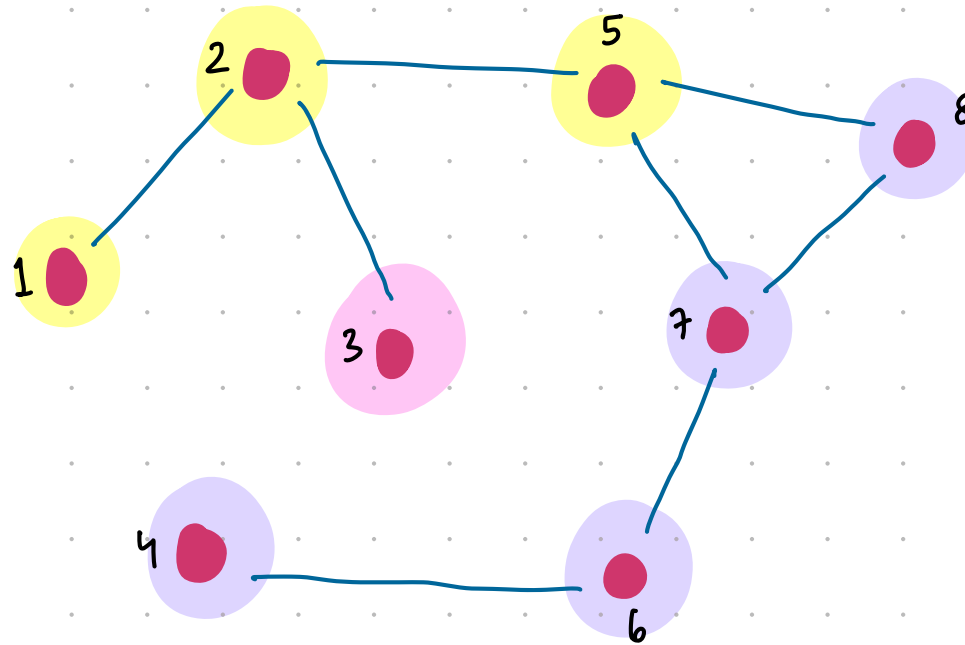
4, 6, 7

DFS



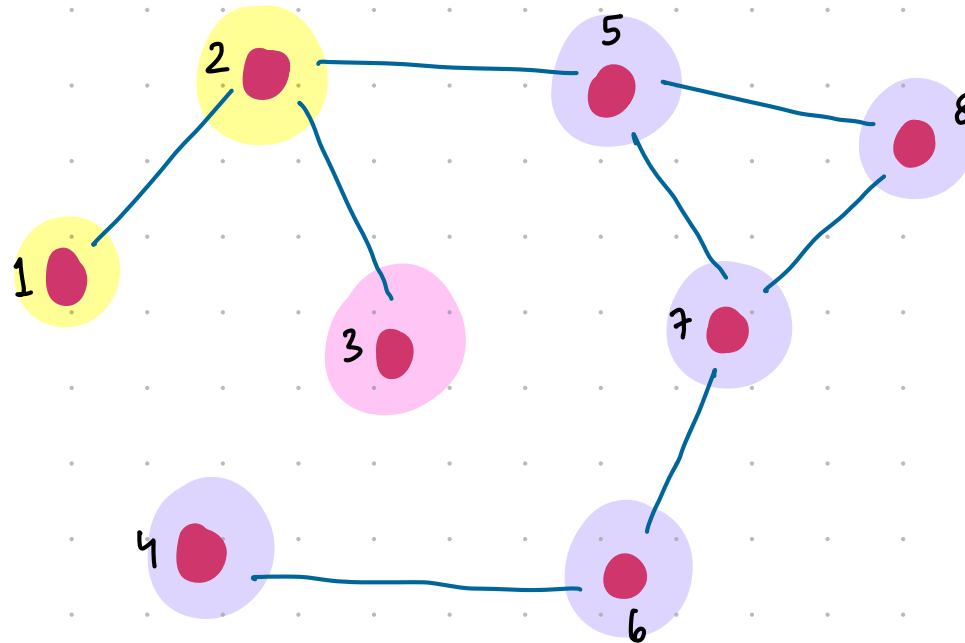
4, 6, 7, 8

DFS



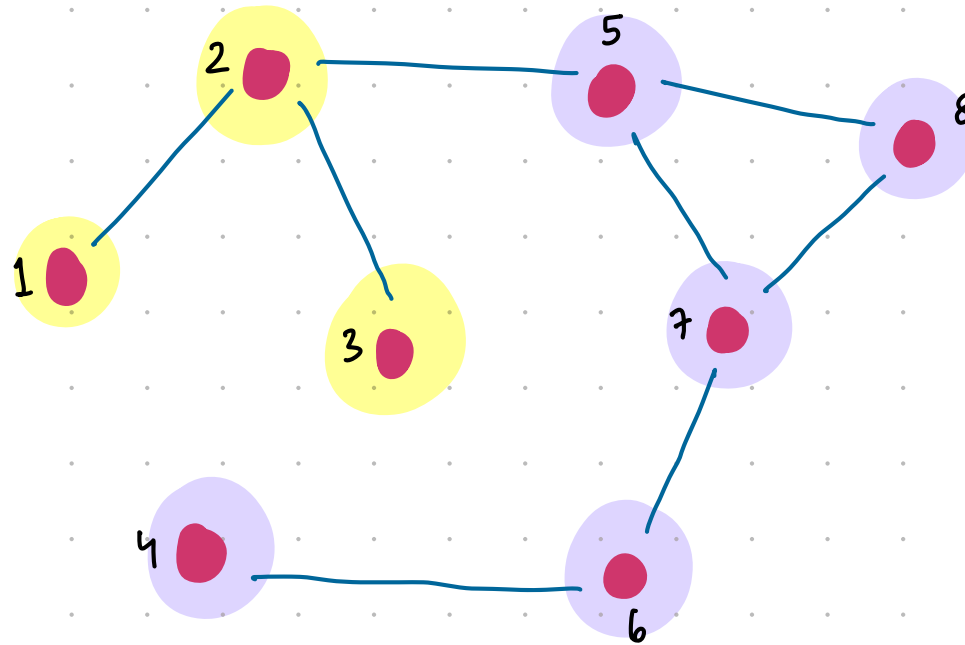
4, 6, 7, 8

DFS



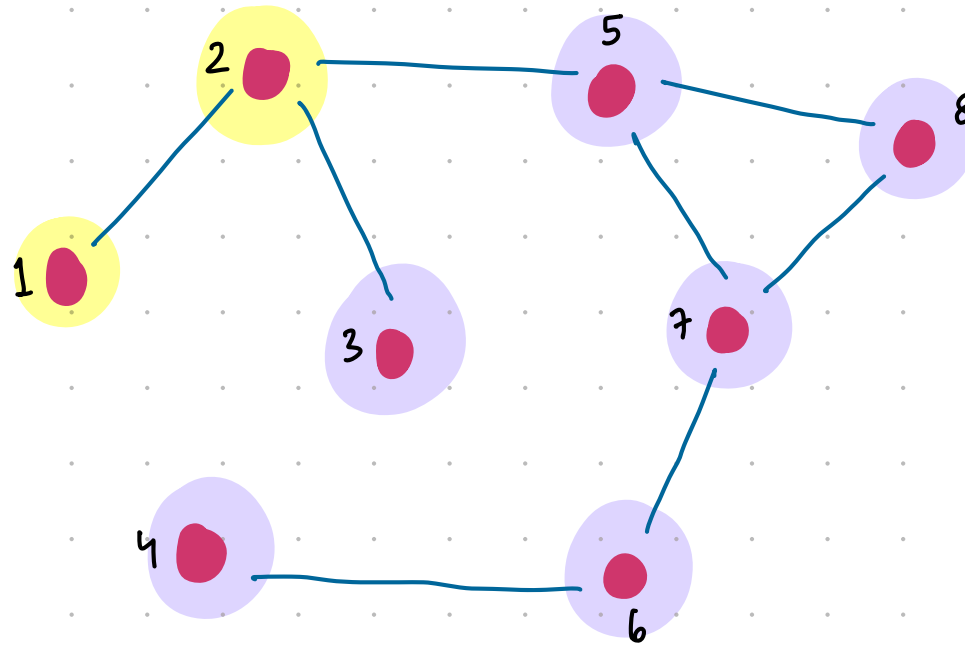
4, 6, 7, 8, 5

DFS



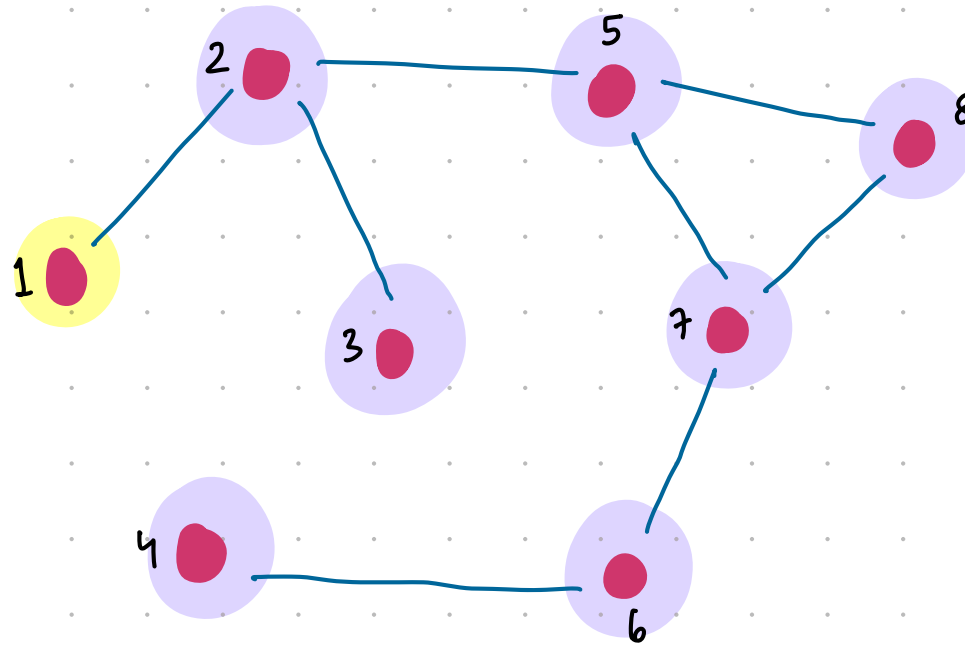
4, 6, 7, 8, 5

DFS



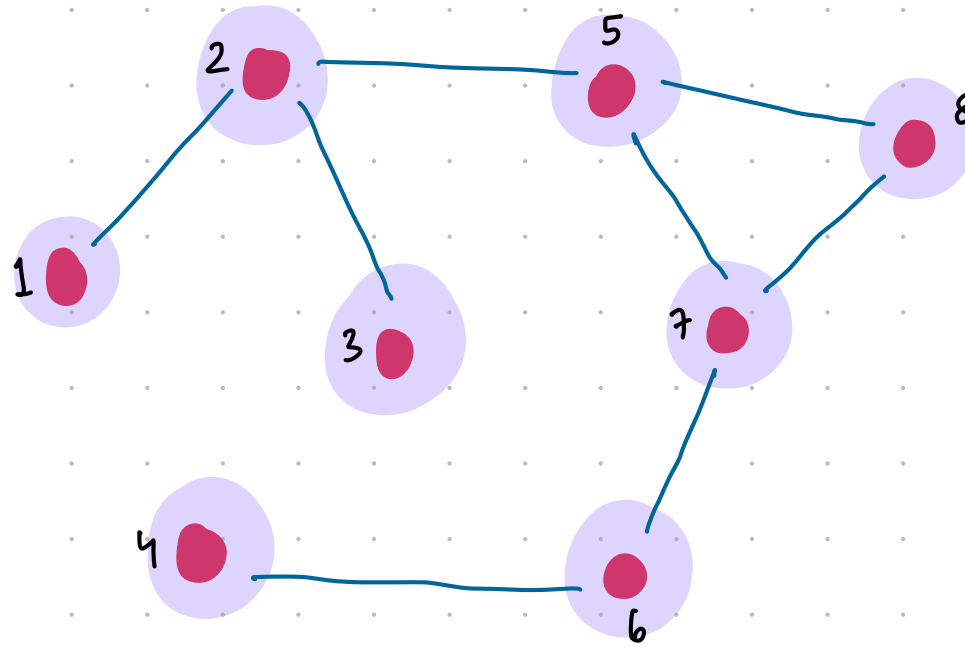
4, 6, 7, 8, 5, 3

DFS



4, 6, 7, 8, 5, 3, 2

DFS



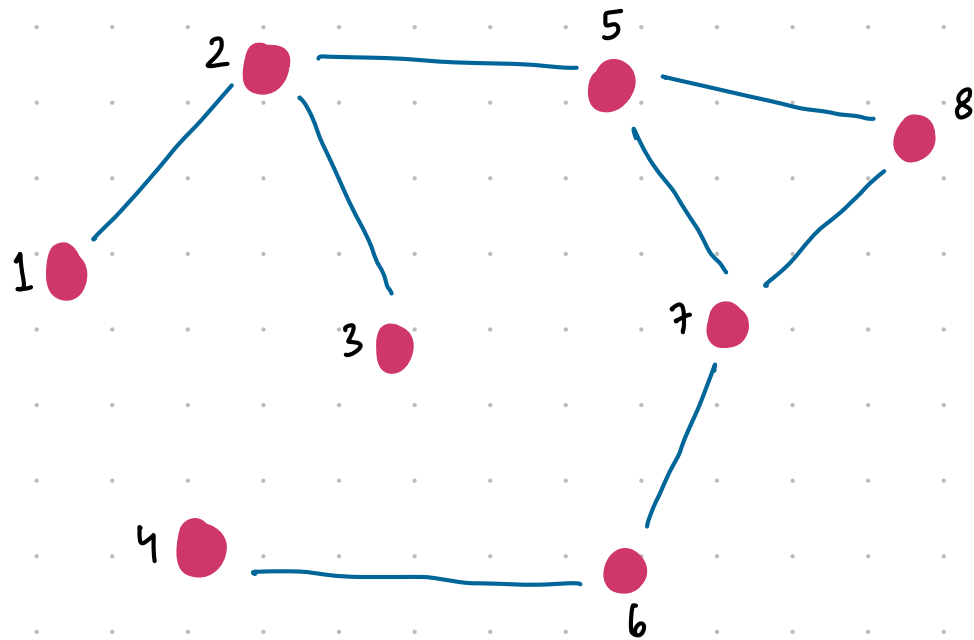
4, 6, 7, 8, 5, 3, 2, 1 |||
0 0 0

BFS

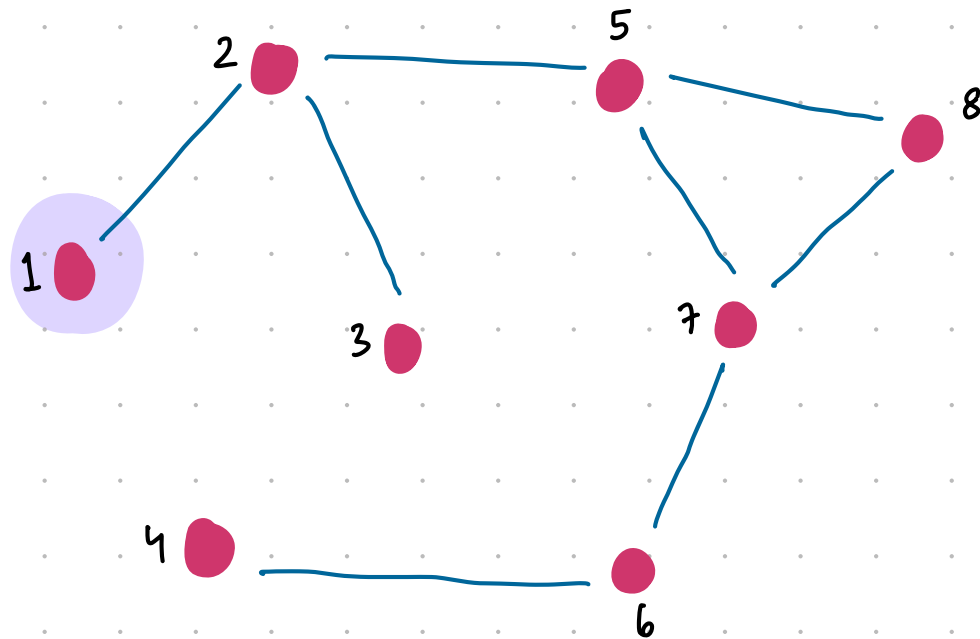
Explore the graph layer-by-layer

```
function bfs(u) {  
    create queue q  
    mark u explored  
    q.enqueue(u)  
    while queue not empty {  
        v = q.dequeue  
        for each unexplored neighbor w of v {  
            mark w explored  
            q.enqueue(w)  
        }  
    }  
}
```

BFS

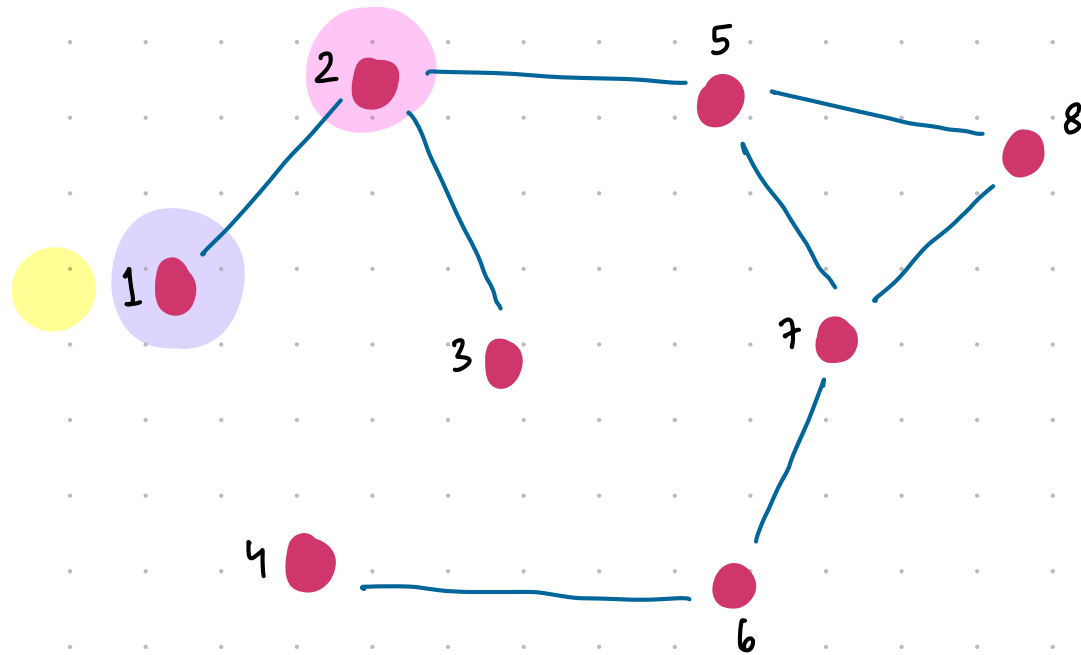


BFS (1)



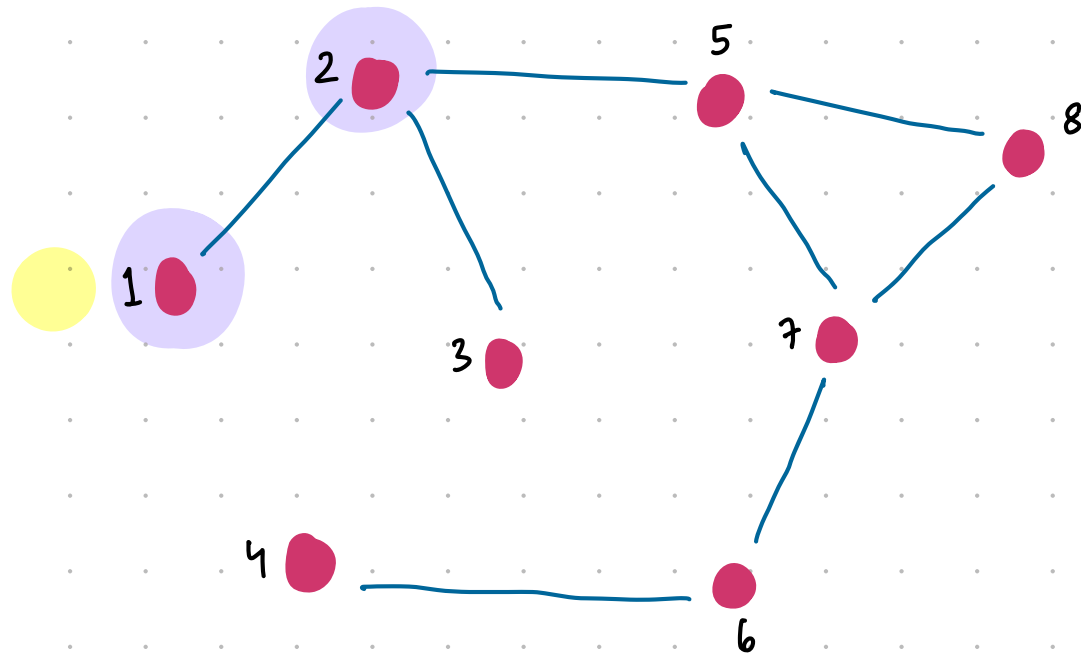
q : (1)

BFS



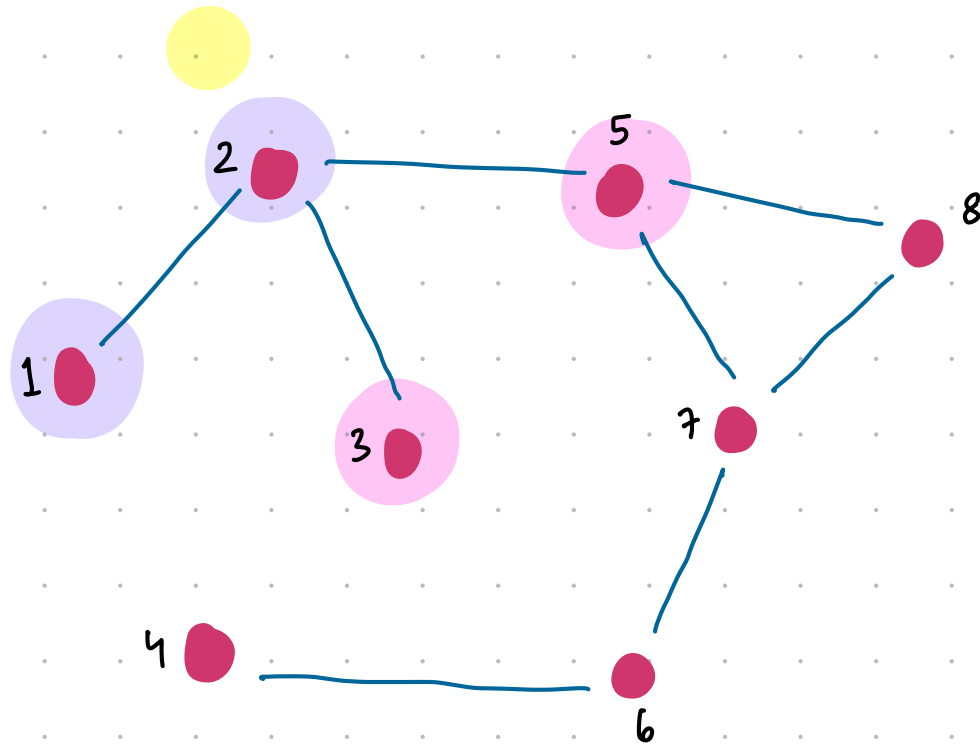
q : (1)

BFS



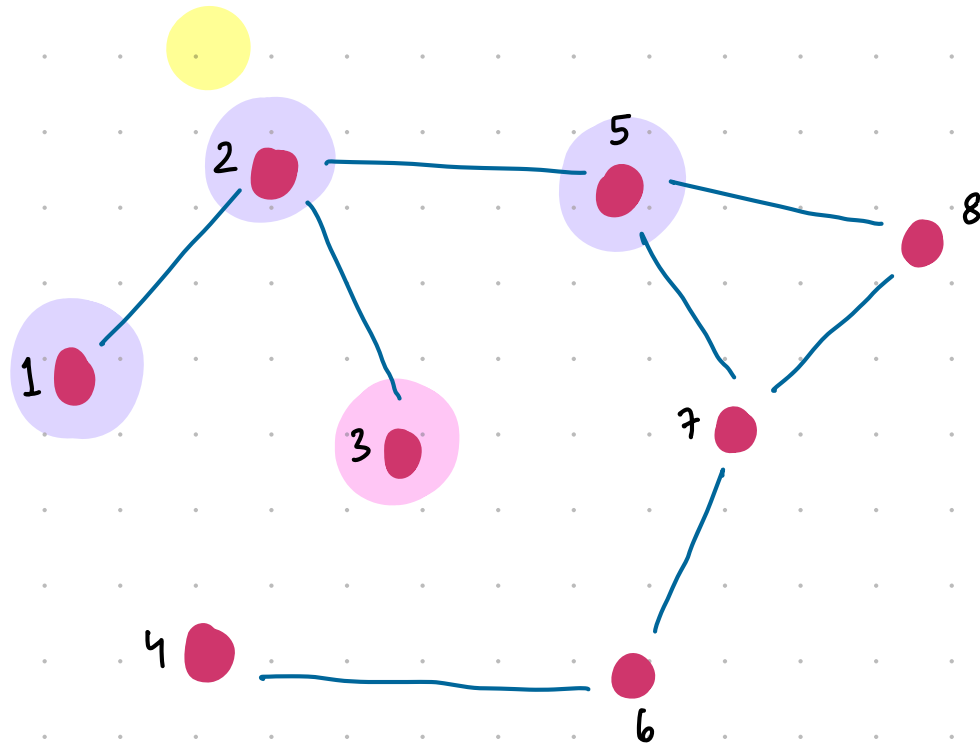
$q : (1, 2)$

BFS



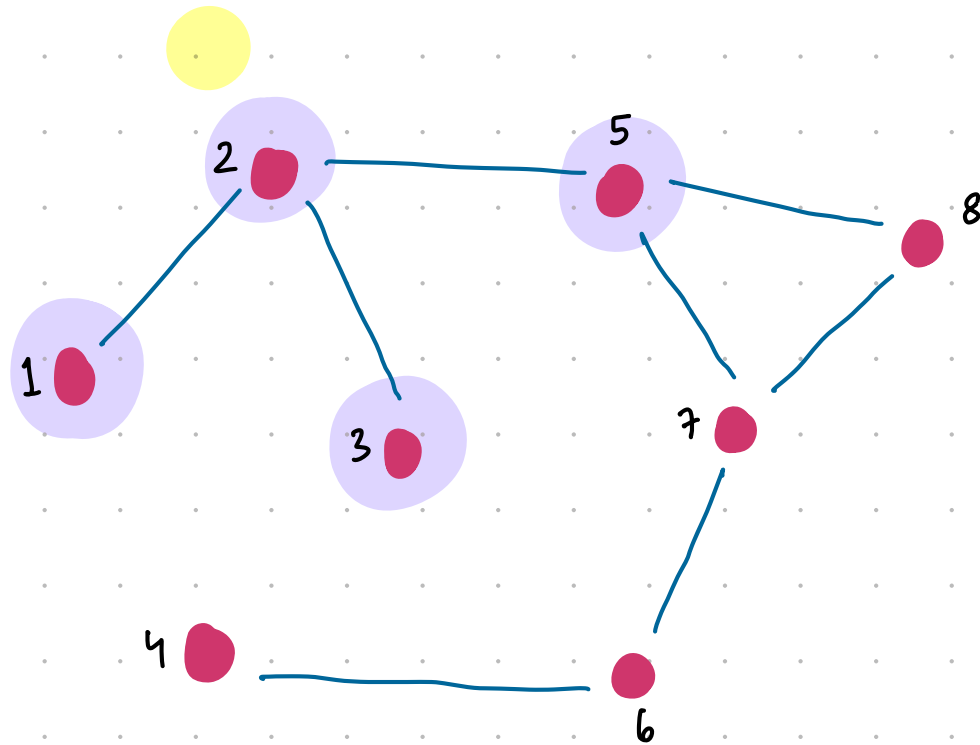
q : (1, 2)

BFS



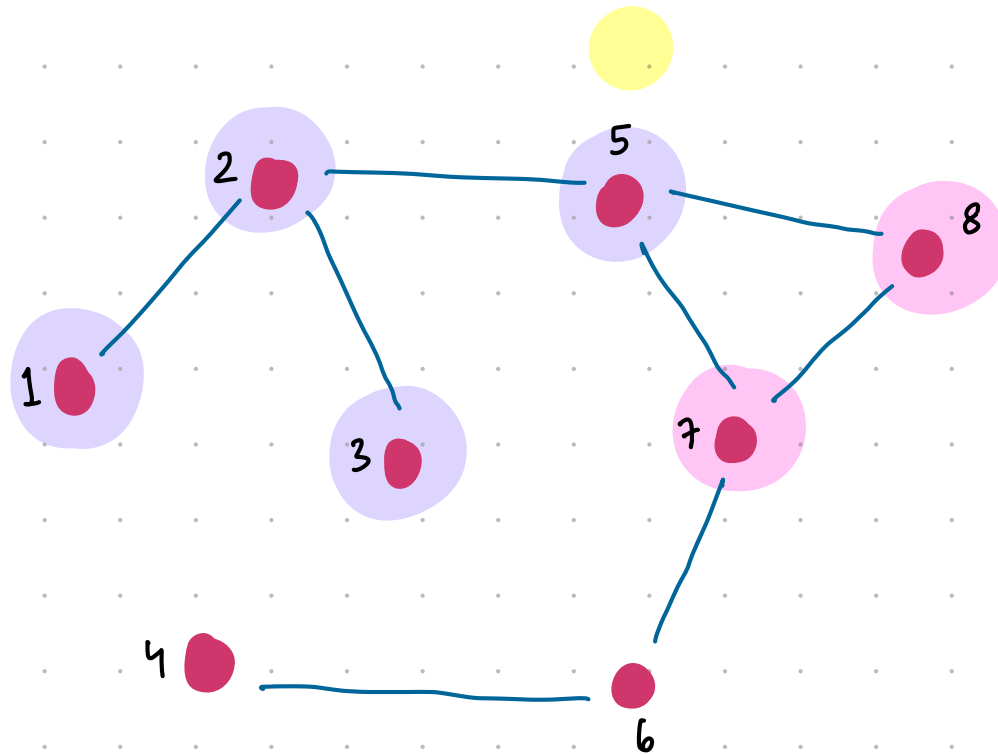
q : (1, 2, 5)

BFS



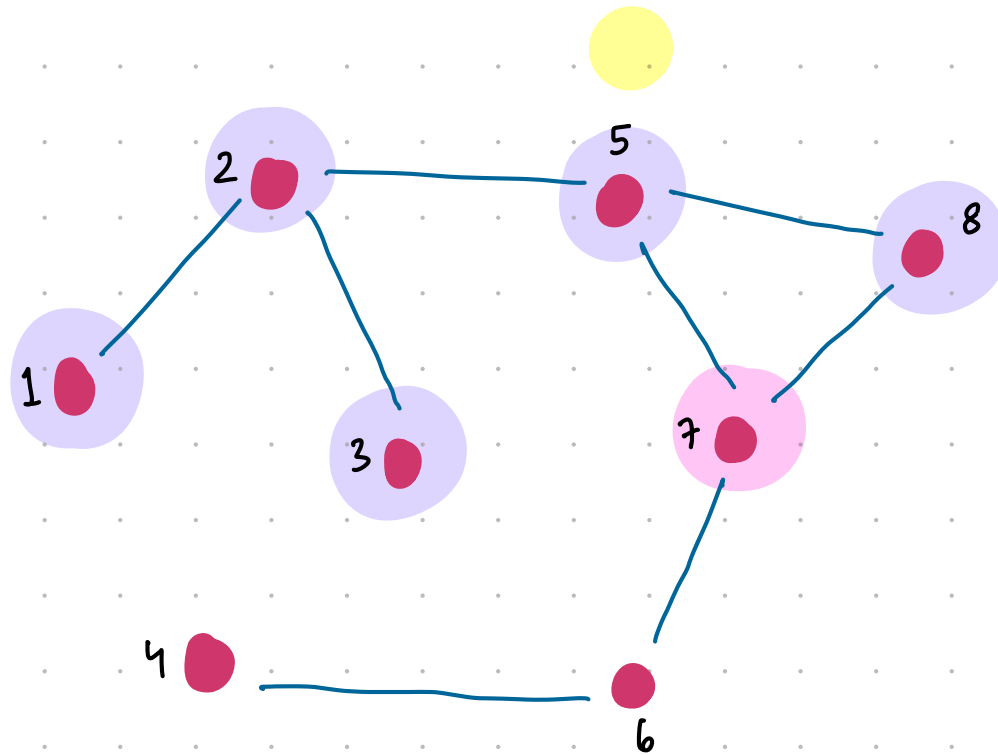
$q : (1, 2, 5, 3)$

BFS



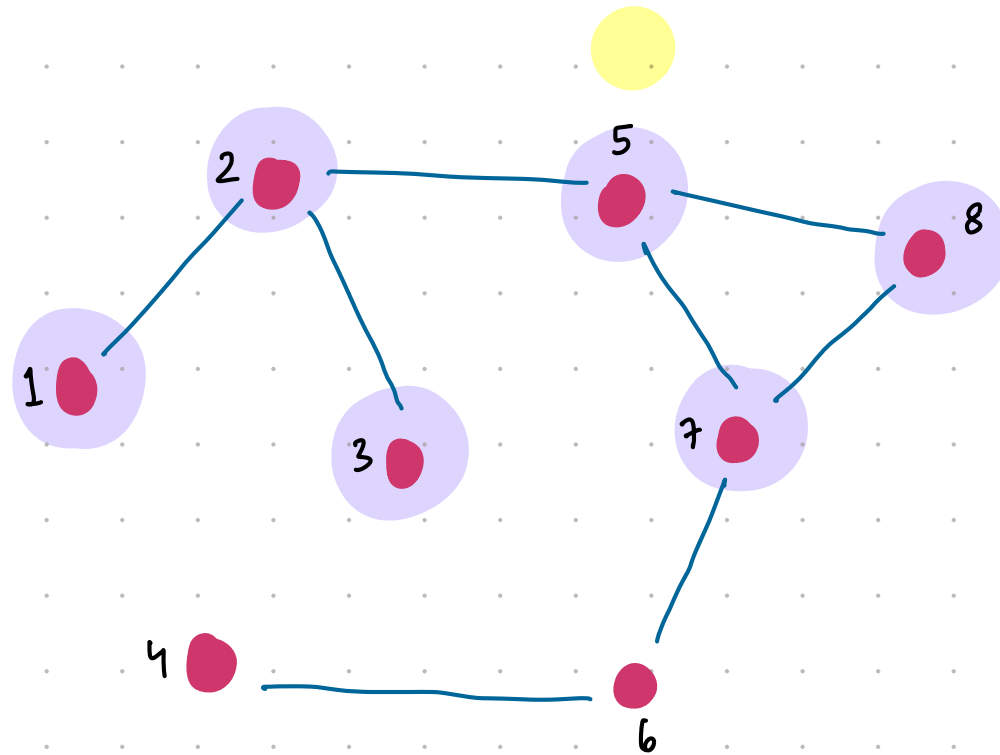
q : (1, 2, 5, 3)

BFS



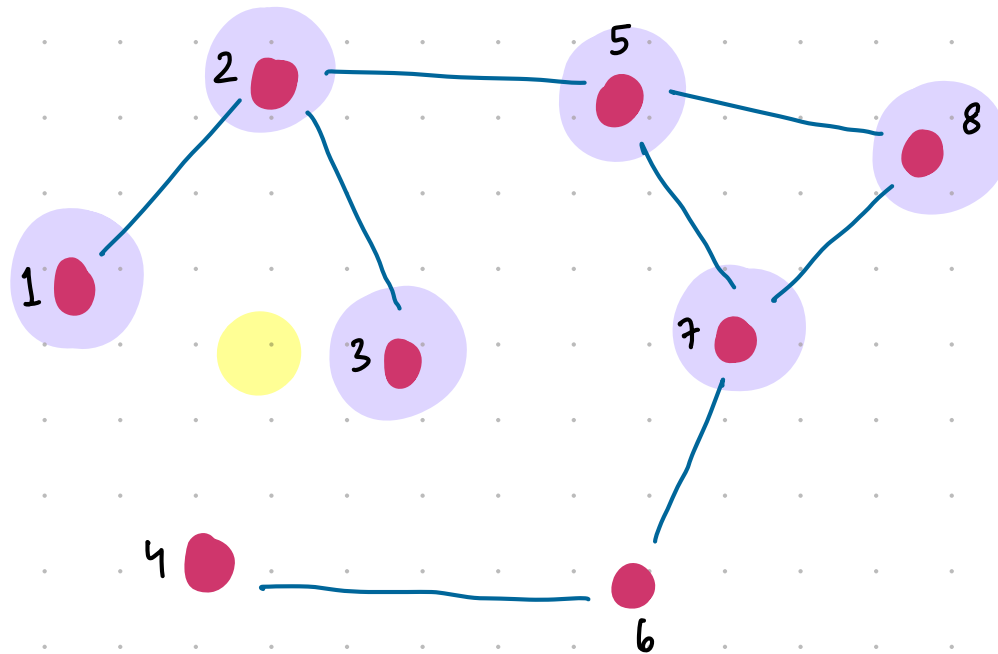
q : (1 , 2 , 5 , 3 , 8)

BFS



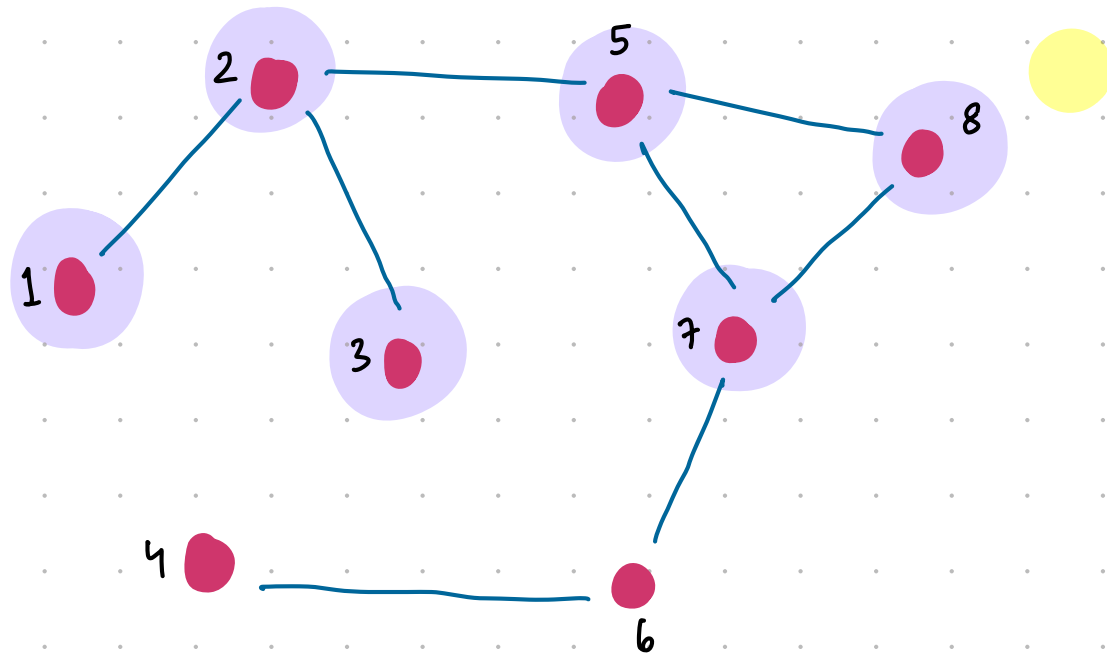
q : (1, 2, 5, 3, 8, 7)

BFS



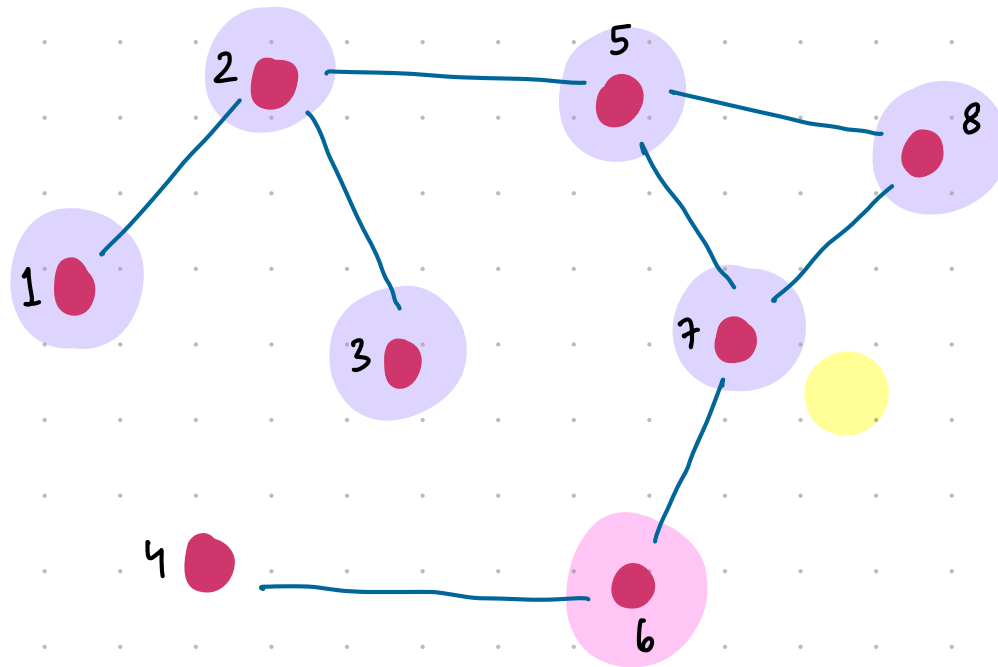
q : (1, 2, 5, 3, 8, 7)

BFS



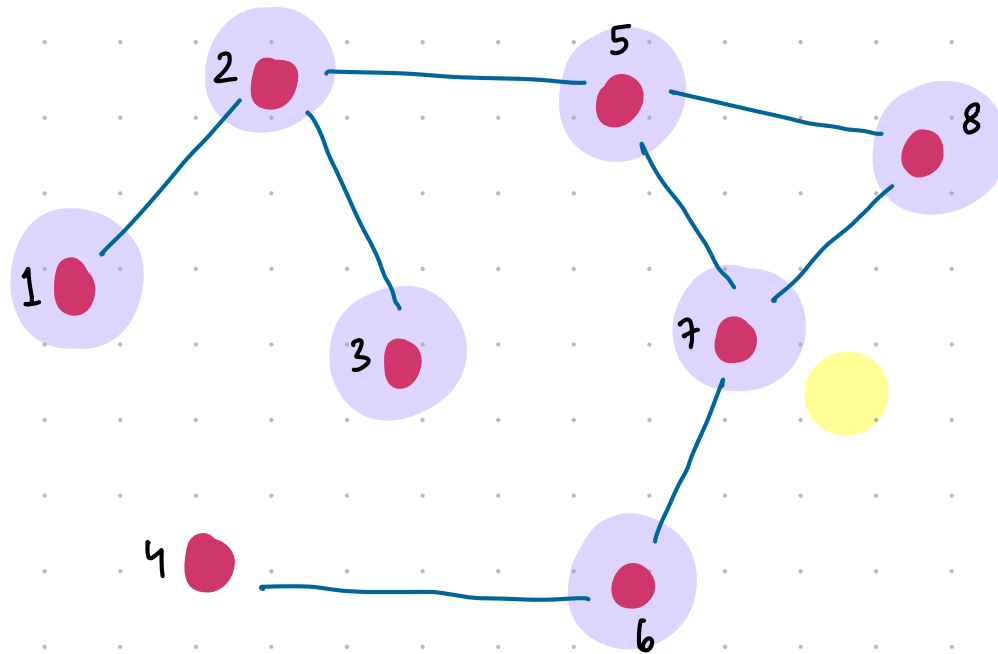
q : (1, 2, 5, 3, 8, 7)

BFS



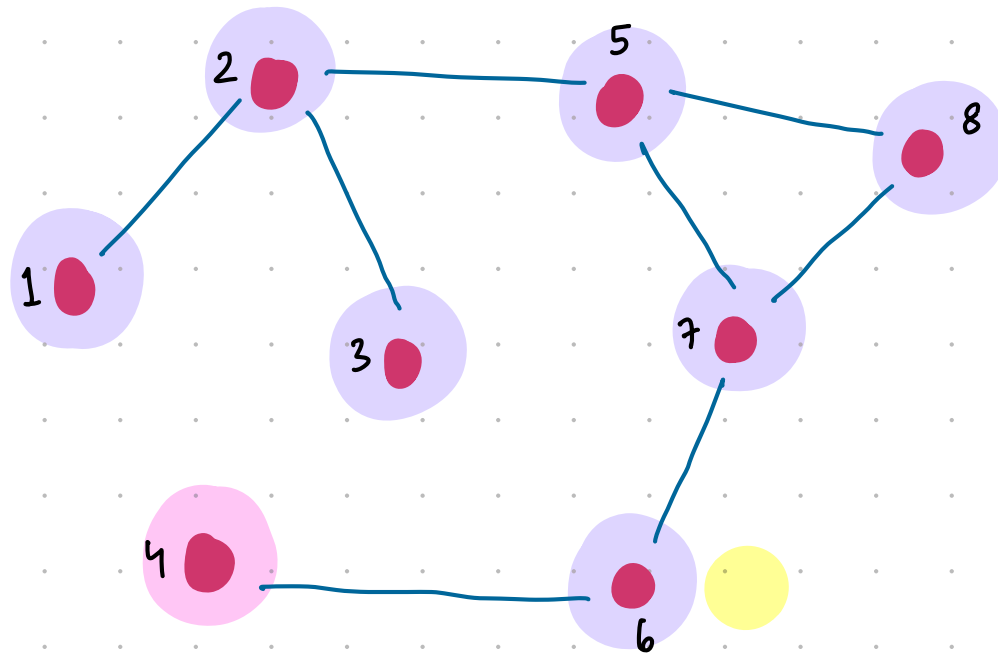
q : (1, 2, 5, 3, 8, 7)

BFS



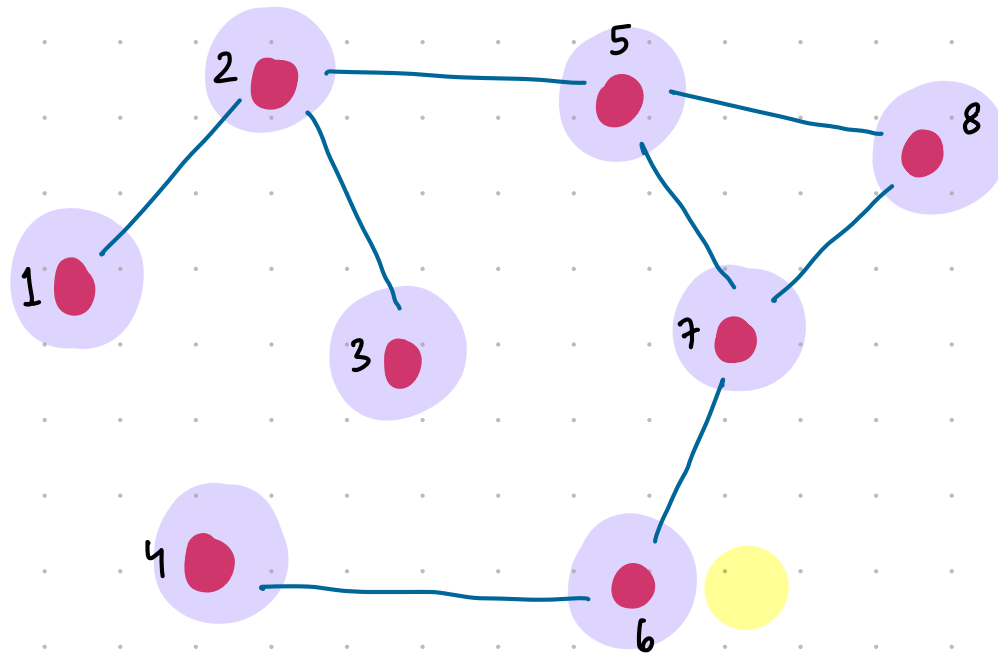
q : (1 , 2 , 5 , 3 , 8 , 7 , 6)

BFS



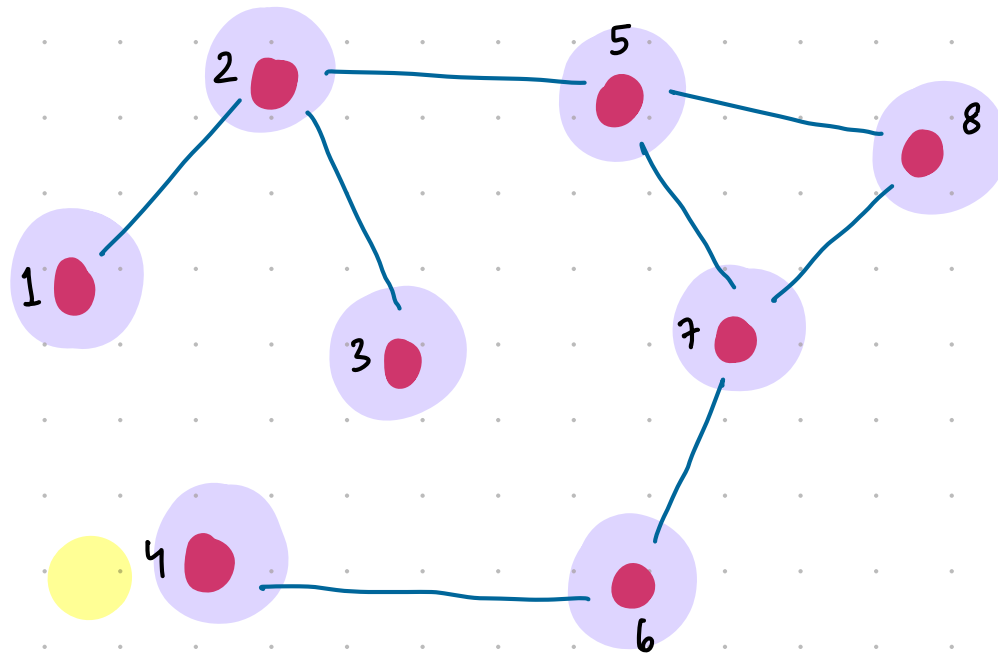
q : (1, 2, 5, 3, 8, 7, 6)

BFS



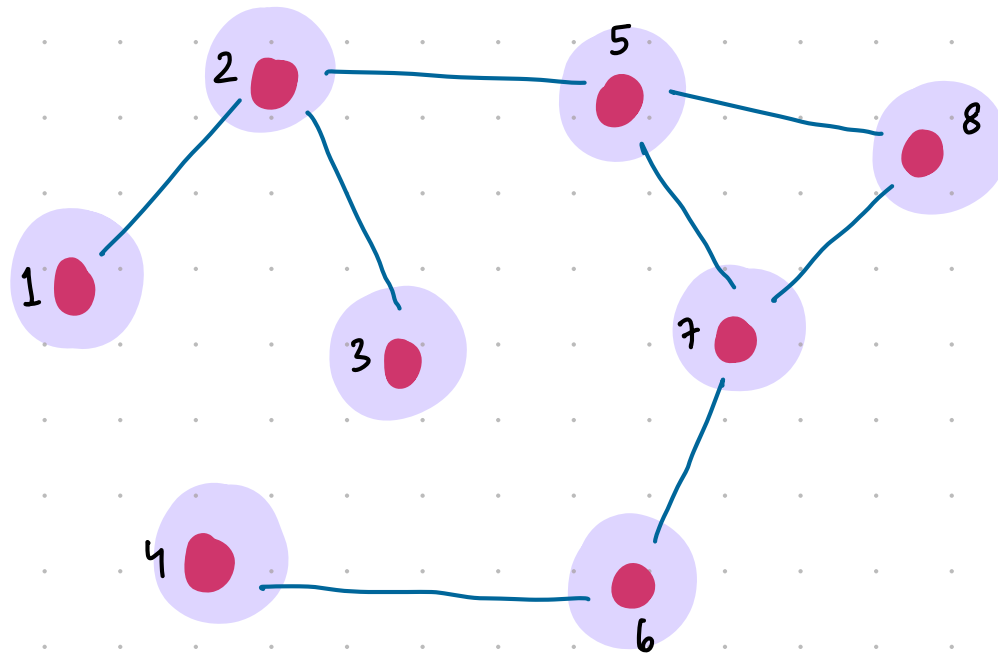
q : (1, 2, 5, 3, 8, 7, 6, 4)

BFS



q : (1, 2, 5, 3, 8, 7, 6, 4)

BFS



q : (1, 2, 5, 3, 8, 7, 6, 4)

We explored all the nodes !

THINGS TO KEEP IN MIND:

- ① Both BFS and DFS explore only the connected component containing the starting vertex.

THINGS TO KEEP IN MIND:

- ① Both BFS and DFS explore only the connected component containing the starting vertex.

We can make very simple modifications to find this:

while there exists an unexplored vertex v {
 dfs(v) // or bfs(v)
}

THINGS TO KEEP IN MIND:

- ① Both BFS and DFS explore only the connected component containing the starting vertex.

We can make very simple modifications to find this:

while there exists an unexplored vertex v {
 dfs(v) // or bfs(v)
}

This fact can be used for finding the number of connected components in a graph.

THINGS TO KEEP IN MIND:

- ① Both BFS and DFS explore only the connected component containing the starting vertex.

We can make very simple modifications to find this:

while there exists an unexplored vertex v {
 dfs(v) // or bfs(v)
}

This fact can be used for finding the number of connected components in a graph.

- ② Both BFS and DFS run in $O(|V| + |E|)$ time and they use $O(|V|)$ memory.